

分 类 号 _____

学号 M201672766

学校代码 10487

密级 _____

华中科技大学

硕士学位论文

基于深度学习的代码漏洞检测

学 位 申 请 人： 张宏浩

学 科 专 业： 网络空间安全

指 导 教 师： 付才 教授

答 辩 日 期： 2019 年 5 月 28 日

**A Thesis Submitted in Partial Fulfillment of the Requirements
for the Degree for the Master of Engineering**

Vulnerability Detection Based on Deep Learning

Candidate : Zhang Honghao

Major : Cyberspace Security

Supervisor : Prof. Fu Cai

Huazhong University of Science & Technology

Wuhan 430074, P.R.China

May, 2019

摘要

软件安全是软件开发工作的重中之重，因为软件安全漏洞不仅会损耗不必要的资源，还会给公司或组织带来严重的经济财产损失。然而对于开发者而言，发现漏洞并不是一件容易的事，因为漏洞代码和安全代码并没有明显的差异。对于代码分析这类时序任务，循环神经网络经大量科研工作者的验证，已经是成熟的应用方案。但是最近有不少的研究将卷积网络用于音频合成和机器翻译等时序任务中，取得了比循环网络更好的效果。

在使用机器学习检测代码漏洞的任务中，代码的预处理和神经网络的选择是影响检测精度的关键因素。论文首先提出了一种将程序代码转换为单词向量序列的方法，该方法可以兼顾保留程序语义和处理的简洁性，包含了代码切割、单词编码等过程。然后建立了一个基于卷积神经网络的代码漏洞检测模型，不同于其他基于循环网络的方法，该模型使用卷积神经网络对单词级的函数对象进行分析。模型结合了多种有效的技术，包括因果卷积、扩张卷积以及残差单元等，并尽量保证网络的简洁。最后，在使用多种超参数对模型的性能进行测试后，给出了兼顾效率与准确度的卷积网络超参数调整原则，为模型处理更多的数据集预留了扩展性。实验用三种数据集来测试模型性能，代码漏洞检测模型在精确率和召回率等多项指标上都取得了极佳的结果。实验结果表明，尽管循环神经网络在时序任务上有更高的理论性能，但基于卷积网络的代码漏洞检测模型在实验各项指标均优于 LSTM 网络，而且卷积网络所需的参数规模更小，这带来了更快的计算速度。

该研究成果让研究人员重新考虑序列模型与循环网络之间的联系，促进了基于卷积网络的序列模型应用的研究。鉴于卷积网络模型在性能和效果上的双重优势，可以预测卷积网络的成功应用并不仅限于特定的领域，在序列模型中应该有卷积网络的一席之地。

关键词：软件安全，漏洞检测，神经网络

Abstract

Software security is a crucial concern within the software development process as software vulnerabilities can not only incur additional costs, but also cause severe damages to an organization or company. For most deep learning researchers, sequence modeling such as software code is synonymous with recurrent networks. Yet recent results indicate that convolutional architectures can outperform recurrent networks on tasks such as audio synthesis and machine translation.

In this paper, we propose a novel approach to vulnerability detection of software based on convolutional neural network, despite most of the existing works are focused on models based on recurrent neural network. The approach relies on the textual analysis of the source code. We describe the convolutional architecture that is applied to vulnerability detection. This architecture is deliberately kept simple, combining some of the best practices of modern convolutional architectures, such as dilated convolutions and residual connections. Ultimately, we test our model using three datasets and achieve very good precision and recall. Our results indicate that the convolutional architecture outperforms canonical recurrent networks such as LSTM, while demonstrating longer effective memory.

We conclude that the common association between sequence modeling and recurrent networks should be reconsidered. The results are very promising and encourage further research in related area. Due to the comparable clarity and simplicity of our model, we conclude that the recent successes of convolutional architectures in applications are not confined to specific domains and convolutional networks should be regarded as a powerful toolkit for sequence modeling.

Key words: Software Security, Vulnerability Detection, Neural Network

目 录

摘 要	I
Abstract	II
1 绪论	
1.1 课题背景及研究意义	(1)
1.2 研究概况	(1)
1.3 研究内容	(6)
1.4 论文组织结构	(8)
2 相关理论和技术	
2.1 传统代码异常检测	(9)
2.2 自然语言处理	(10)
2.3 循环神经网络	(10)
2.4 卷积神经网络	(12)
2.5 本章小结	(12)
3 基于深度学习的代码漏洞检测模型	
3.1 模型总体架构	(13)
3.2 代码编码预处理	(14)
3.3 时序卷积神经网络	(17)
3.4 时序卷积神经网络调参准则	(23)
3.5 本章小结	(24)
4 代码漏洞检测模型的实现	
4.1 代码编码预处理	(26)
4.2 时序卷积神经网络	(28)
4.3 本章小结	(31)

5	模型测试与结果分析	
5.1	实验环境.....	(32)
5.2	实验数据.....	(32)
5.3	实验结果及分析.....	(33)
5.4	本章小结.....	(43)
6	总结与展望	
6.1	工作总结.....	(44)
6.2	工作展望.....	(45)
	致谢.....	(46)
	参考文献.....	(47)

1 绪论

1.1 课题背景及研究意义

在大数据的时代背景下，不仅是用户数据迎来了海量的增长，计算机行业的代码量也以惊人的速度在扩张。仅 2017 年，国外著名代码托管网站 GitHub 就收到了包含 337 种不同编程语言的 10 亿次代码提交。因此，代码筛选、代码分类和代码语义分析成为各个领域极其重要的任务，这些任务可以实现包括代码查重、性能预测、代码漏洞检测等一系列有价值的功能。然而，代码的复杂多变性使得这些工作变得极具挑战。因为在保持语义不变的前提下，代码可通过多种方式在语法层面上进行修改。例如，开发人员可以重排一些语句，甚至直接使用不同的语言编写程序，毕竟不同的语言有着不同的语法规则。但是机器学习的出现提供了新的解决思路，因为这些问题适用于典型的回归模型和分类模型。

每年在软件中都有数以千计的安全漏洞被发现，这些漏洞有的被报告给通用漏洞披露数据库（Common Vulnerabilities and Exposures, CVE），更多的则是发现于内部专用代码中，这使得它们在曝光给公众之前得以修复。尽管漏洞代码会对程序的安全性和稳定性造成极大的损害，但在通常情况下漏洞代码和安全代码的区别并不明显，这导致开发人员无法在第一时间发现漏洞。如果基于机器学习对海量的源代码进行分析，就有可能发现漏洞代码的错误模式，从而让计算机帮助开发人员识别漏洞。

1.2 研究概况

1.2.1 基于 RNN 的序列模型

循环神经网络（Recurrent Neural Network, RNN）的产生就是为了处理有着长期依赖的时间序列问题，所以对基于循环神经网络的序列模型有着诸多的研究。由于在循环网络的设计中存在很大的自由度，所以评估各种循环网络结构成为一项研

究热点内容。Chung 等人^[1]在复调音乐建模和语言信号建模的任务上评估了 LSTM（Long Short-Term Memory）与 GRU（Gated Recurrent Unit）的性能，结果表明这两类复杂的门控单元比传统的 RNN 单元更好，而且 GRU 在简化了 LSTM 的同时，性能却可与后者相媲美；Pascanu 等人^[2]尝试了不同的构建深度 RNN 的方法，并且比较了不同架构在复调音乐建模、字符级的语言模型和单词级的语言模型上的表现，结果表明深度 RNN 优于传统浅层 RNN。Jozefowicz 等人^[3]的研究对于 RNN 的研究人员是极具吸引力的，他们整理了一万多种不同的 RNN 架构，并且比较了它们在多种任务中的表现，结果表明，尽管可以针对特定的任务找到优于 LSTM 和 GRU 的结构，却没有一种结构可以在所有任务中优于 LSTMs，所以 LSTMs 仍是容易找到的最好的结构。Greff 等人^[4]的研究专注于 LSTM 的各种变体，他们收集了 8 种 LSTM 的变体，并对它们在语音识别、手写识别和复调音乐建模上进行了基准测试，最终发现没有任何一种变体可以显著改进标准 LSTM 结构，并且发现遗忘门和输出激活是 LSTM 中最关键的结构；Zhang 等人^[5]系统地分析了 RNN 的连接体系结构，并且评估了它们在特征级语言建模和综合压力测试方面的性能，发现深层的循环网络有着更好的性能；Melis 等人^[6]在单词级语言模型和字符级语言模型上测试了多种 LSTM 的变体，最终得出结论：LSTM 及其变体在正确规范化的前提下优于更新颖的模型。

从以上诸多研究成果中可以看到，LSTM 及其变体在时序任务中有着非常优秀的表现，它们几乎作为循环神经网络应用的默认选择，然而循环神经网络的固有缺陷，即内存消耗大、网络难以训练、易发生梯度消失或梯度爆炸的缺陷，却阻碍了其发展。而本文最终的实验也选择与 LSTM 网络作为对比。

1.2.2 基于 CNN 的序列模型

卷积神经网络（Convolutional Neural Network，CNN）由 LeCun 等人^[7]在 1989 年首次提出，并成功应用于美国邮政局提供的手写邮政编码的识别。同年，Hinton 等人^[8]将其应用于小型序列任务。Waibel 等人^[9]将卷积网络用于英语音素的识别，

并与离散的隐马尔可夫模型对比,结果显示卷积网络的准确率更优。Bottou 等人^[10]在以前工作的基础上,将卷积网络用于法语的数字识别。

而让卷积网络真正大展拳脚的领域,仍然是自然语言处理(Natural Language Processing, NLP)。Kalchbrenner 等人^[11]构建了一个名为动态卷积神经网络的模型,并在二分类和多分类的情感预测任务中实现了出色的性能。Kim^[12]实现了一个简单的 CNN 用于句子分类任务,只需要很少的参数调整便在包括情感分析和问题分类在内的多个任务中取得了最佳的结果。Zhang 等人^[5]对于字符级卷积神经网络的文本分类进行了研究,与传统模型(如词袋、n-gram)和其他深度学习模型(如单词级卷积网络和循环网络)相比取得了更好的结果。Conneau 等人^[13]提出了一个用于文本分类的新架构,名为 VDCNN,它使用小卷积和池化操作以及深度达到 29 层的卷积层,刷新了几个公共文本分类任务的最佳结果,这也是将极深卷积网络用于文本分类任务的首次尝试。Johnson 和 Zhang^[14]在文本分类问题上没有采用常规的低维词向量作为输入,而是另辟蹊径地直接应用于高维文本数据,并结合卷积层中的词袋转换,结果与最先进的方法相比也不落下风。

Kalchbrenner 等人^[15]提出了一种新的序列处理模型,名为 ByteNet,它是一个包含编码器和解码器的一维卷积神经网络,在字符级语言建模和英语到德语的翻译上取得了历史最佳结果,碾压了同期的循环神经网络。Gehring 等人^[16]提出了一种用于机器翻译的卷积神经网络模型,并在多个翻译任务中与双向 LSTM 网络进行对比,在保持结果相近或更好的基础上,将运算速度提高了两倍以上。Van den Oord 等人^[17]提出了一种生成原始音频的深度卷积网络,名为 WaveNet,它在文本语音转换、音乐片段生成和音素识别等多个任务中均有着极佳的表现。Dauphin 等人提出了一种语言建模的深度卷积模型,并引进了一种新的简化门控单元,在 WikiText-103 数据集上不仅刷新了最佳结果,而且与循环网络相比,将延迟降低了一个数量级。

从以上诸多研究成果中可以看到,卷积神经网络在自然语言处理的各项任务中取得了显著成效,尽管还没有直接将卷积网络用于代码漏洞检测的研究,但自然语言与编程语言的相似性使得将卷积网络应用于编程语言分析任务成为可能。

1.2.3 代码编码预处理

代码编码处理是使用机器学习方法分析代码的必要过程。一方面，机器学习算法只接收数值的输入，程序代码无法直接输入神经网络模型。另一方面，将代码编码为数值必须反映单词之间的关联性，著名的 one-hot 编码方式尽管简单易懂，却也有着维度过高、无法反映单词之间关系的缺点。代码处理的通常思路是，先把输入代码处理成令牌，然后将令牌嵌入到连续的低维空间中。词嵌入的基本思想即是根据令牌之间相似性来学习令牌到低维空间的映射。

Hoa 等人^[18]以 Java 为原始语言做令牌的词嵌入，并使用 LSTM 网络建立了语言模型。Gu 等人^[19]构建了一个名为 DeepAPI 的卷积网络模型，以 Java 为原始语言做编码，用来生成 API 的使用顺序，并取得了良好的效果。Dor 等人^[20]提出一种神经网络模型用于将源代码和编译目标代码的语句对齐任务，他们将 C 代码语句和目标代码语句做嵌入，做到了高精度的对齐。Chris 等人^[21]以 OpenCL 为原始语言做令牌词嵌入，他们的神经网络模型在预测异构并行化的最佳映射和 GPU 线程化因子的任务中取得了良好效果。

Miltiadis 等人^[22]将变量名嵌入到高维空间中，实现了变量、方法、甚至类的自动化命名。Uri 等人^[23]将代码片段表示为固定长度的代码向量，从而学习代码段的语义属性，他们通过预测方法的名称来证明其模型的有效性。Martin 等人^[24]构建了一个用于代码克隆检测的深度学习框架，支持对单词级和语义级的代码分析，并在真实的 Java 系统中证明其方法的有效性。Veselin 等人^[25]将代码补全问题转换为预测句子概率的自然语言模型，从而实现了 API 代码补全。Yang 等人^[26]提出了一种名为 PCC 的方法来优化基于令牌的语言模型，使模型可以处理长令牌序列，并在 Java 中证明了方法的有效性。Miltiadis 等人^[27]结合注意力机制，将神经网络应用于代码摘要任务中，并在多个 Java 项目中证明了其网络的性能。Mou 等人^[28]提出了一种名为 TBCNN 的基于树的神经网络模型，其可以捕获抽象语法树上的结构信息，从而完成对代码的分类任务。

当前的代码处理方式结合了多种机制，但处理方式相对复杂，而且大都仅适用于单一的数据源，缺乏通用性，阻碍了模型处理更多数据的可能。

1.2.4 基于机器学习的漏洞检测

除了传统的静态和动态分析方法，将机器学习用于代码漏洞检测是当前非常热门的领域。Hovsepyan 等人^[29]将 Java 源代码转换为 token 并用词袋表示，然后使用支持向量机（Support Vector Machine, SVM）进行分析，并将结果与静态分析工具进行对比，取得了不错的效果。Pang 等人^[30]对此项技术进行了扩展，结合了 n-gram 分析和特征选择算法，并使用 Android 应用程序对模型进行评估，取得了很高的精度。

Abou 等人^[31]使用基于字节级的 n-gram 分析方法检测恶意代码，在训练数据和交叉验证中取得了极高的准确率。Silvio 等人^[32]将恶意软件的控制流图作为其签名，并使用基于特征向量之间的距离来度量签名之间的相似性，从而检测恶意软件，与其他算法相比，他们的算法可以发现更多的恶意软件变体。Jeremy 等人^[33]综合了机器学习和数据挖掘技术来检测恶意可执行程序，使用 n-gram 字节编码作为特征，并使用决策树进行归纳，最终结果达到了 0.996 的 ROC 面积。Asaf 等人^[34]通过训练对 OpCode 和字节级 n-gram 的分类器，然后对分类器的结果应用加权算法和主动学习机制，达到了相当高的检测率。

Hsiao 等人^[35]基于 280 万个 JavaScript 程序构建 n-gram 模型，并将代码模型描述为程序依赖图的子图，进而度量不同代码行的重要性，他们通过实现一个抄袭检测器来证明该模型的能力。Veselin 等人^[36]将程序代码转换为中间表示，将程序属性预测转换为机器学习的结构化预测工作，并实现了一个名为 JSNice 的预测引擎，在预测 JavaScript 标识符名称和类型注释上取得了良好的效果。Martin 等人^[37]结合深度学习模型和 Java 项目的 n-gram 模型，并将其应用于代码建议任务，表明了模型在实际软件工程任务中的可用性。Zheng 等人^[38]构建了一个名为 EKLAVYA 的递归神经网络，从反汇编的二进制代码中恢复函数类型的特征，并且结果不依赖于指令集、编译器或优化等级。Fabian 等人^[39]通过从代码中提取抽象语法树，并确定树中的结构模式，使得代码中的每个函数可以用这些模式来描述，他们在开源项目代码中评估该方法，只需要用少量的样本训练即可识别 0-day 漏洞。

还有很多研究工作已经不止步于漏洞的检测，进而研究漏洞的修复。Claire 等人^[40]提出了一种名为 GenProg 的模型，使用遗传编程的扩展形式，并且不受特定漏洞类型的影响，在不引入新的缺陷的前提下，达到了代码漏洞修复的目的。Fan 等人^[41]提出了一种名为 Prophet 的模型，它生成候选修复方案的空间，并按照正确的可能性进行排序，然后用测试用例验证修复方案以找到正确的补丁，他们在 8 个开源项目的测试中取得了良好的效果。Gupta 等人^[42]构建了一个名为 DeepFix 的多层 seq2seq 神经网络，并将其训练预测程序语法错误的位置以及所需的正确语句，最终将其应用于学生代码的简单语法修复工作。Harer 等人^[43]使用生成对抗网络来做代码修复工作，他们通过从一个离散域映射到另一个目标域，摆脱了需要成对的标记样本的限制条件。

1.3 研究内容

对代码进行异常检测首先面临的问题就是代码的编码问题，因为机器学习算法仅接收数值输入，而如何对代码进行编码是一项实践性很高的工作。尽管有诸多对代码编码方法的研究，但是要评价各种方法的优劣，仍然需要实践来证明。机器学习算法的选择也是一项重要的问题，由于循环神经网络在时间序列问题上的天然优势，基于循环网络的代码分析研究工作相对较多，而且都取得了很好的结果。然而循环网络也有其固有的缺陷，例如预测无法并行处理，后面时间步的结果必须等待前任完成，或是梯度不稳定的问题，容易发生梯度消失或梯度爆炸。这些缺陷都让研究人员重新思考如何在取得较高准确率的前提下，让检测模型的适用面更广、训练限制条件更宽泛、消耗资源更少。同时，近几年将卷积网络应用于序列任务的研究也在逐渐增加，并有不少的应用都取得了极佳的结果，这些成果提供了代码漏洞检测的另一种可能，可以基于卷积网络进行漏洞检测工作。在这些思考的前提下，要实现一个代码异常检测模型所面临的问题有如下几点：

(1) 如何处理代码，既能够保留足够的信息来判断漏洞，又可以尽量减少冗余信息。因为冗余信息不仅浪费计算资源，而且对漏洞判断造成干扰。代码数据预处理方法的优劣会直接影响深度学习的效果。

(2) 如何使用卷积神经网络处理时间序列信息。因为程序代码是一种语言模型，在预测某一时刻的结果时可以利用过去的信息，但是不能利用未来的信息进行漏洞判断，所以要在时序上控制模型对于信息的采集。

(3) 如何使用卷积神经网络进行长序列的学习，同时控制模型的规模。通常一个深而广的模型意味着巨量的计算资源和存储资源的消耗，在处理长序列时如果不控制模型的规模，可能会造成模型的可用而不实用。

本文提出一种使用卷积神经网络模型来进行漏洞检测，并将其应用于两类 CWE (Common Weakness Enumeration) 数据集及混合数据集。实验使用的数据集是 Li 等人^[44]进行部分预处理后的数据，数据的形式是 C/C++ 函数中部分代码行，它们在语义上彼此相关联，即在数据或控制上有相互依赖。然后将代码数据进行编码，并输入模型进行训练。实验结果表明基于卷积网络的代码漏洞检测模型在两类 CWE 数据集以及混合数据集中都有着更好的表现，而且其规模更小，训练速度更快，同时代码处理方式也更简洁。本文的主要贡献有：

(1) 提出一种将程序代码转换为单词向量序列的方案。为了在保留程序语义的前提下将程序代码转换为神经网络的输入，首先对原始代码进行精简，保留必要信息的同时缩小代码规模，然后使用词法分析器把关键字和变量编码为向量，并生成向量序列，作为网络的输入。

(2) 建立一个基于卷积神经网络的代码漏洞检测模型，结合了因果卷积、扩张卷积及残差单元等技术，使得模型能够处理具有长期依赖的代码序列，并保证训练的稳定性。因果卷积在某时刻的输出仅和前一层该时刻之前的元素做卷积，防止信息从未来向过去的泄漏，使得模型适用于时序任务。扩张卷积在达到相同的感受野的前提下，大大降低模型的深度和广度，残差单元的使用可以稳定深度模型的训练，同时提高了检测的准确率。

(3) 给出如何兼顾效率与准确度的卷积网络超参数调整原则，为模型处理更多的数据集预留了扩展性。在实验中探究了卷积网络的超参数调整思路，以及不同超参数对于网络准确率的影响，最终给出针对特定数据集时卷积网络的超参数调整准则。

1.4 论文组织结构

本文由六个章节组成，其组织结构如下所述：

第一章内容是研究工作绪论，主要是理论基础与研究内容的概述。首先介绍了国内外基于机器学习的时间序列问题的研究与应用，以及对于程序代码处理的相关方法。然后简要介绍了论文的研究内容，即需要解决的问题以及论文贡献点，最后给出了论文组织结构。

第二章内容是代码漏洞检测相关的理论及技术介绍，包括传统的代码漏洞检测方法、自然语言处理相关理论、循环神经网络和卷积神经网络在时序任务或代码分析任务中的可能性，作为之后章节的技术铺垫。

第三章介绍了基于卷积神经网络的代码漏洞检测模型，从总体架构设计到代码编码预处理方案，再到卷积神经网络模型的设计，最后给出了模型超参数调整准则，结合理论说明了各个模块的设计方案及设计用意。

第四章介绍了代码漏洞检测模型的实现方法，从数据预处理到模型数据加载，最后是模型的构建，并给出了关键模块的程序处理流程图，表明了从设计到实现的基本思路。

第五章对代码漏洞检测模型的性能进行了测试，在三种数据集上测试模型的各项性能指标，通过与其他成果的对比证明了代码漏洞检测模型的优越性。

第六章对本文进行了总结和展望，首先总结了本文的研究成果及贡献，然后分析了本文所实现的代码漏洞检测方法的不足，并展望了未来的研究方向。

2 相关理论和技术

2.1 传统代码异常检测

软件安全漏洞通常是由代码中的隐藏缺陷导致的，攻击者可能破坏程序正常的控制流并使其执行预期之外的行为，比如让程序崩溃或泄漏用户敏感信息。每年都有数以千计的漏洞被报告给通用漏洞披露数据库，尽管这些漏洞的暴露已经足够触目惊心，但还有更多的漏洞在内部代码中发现并修复^[45]，这些内部漏洞也是软件安全的隐患。近几年有不少关于软件漏洞的报道，例如心脏出血漏洞^[46]和 Equifax 公司数据库泄漏^[47]，这些漏洞会造成巨大的经济损失并对用户的信息安全造成严重的威胁，而让行业安全从业人员不得不重视。软件漏洞的产生往往是由于开发人员的疏忽，而且由于开源软件的流行和代码重用的原因，使得漏洞在计算机软件领域内迅速传播。而且漏洞代码和安全代码的相似性，使其无法直观地被开发人员发现并修复。尽管已经有静态和动态分析程序漏洞的工具，但是这些工具通常仅支持基于预定义规则的检测，其受限于专家经验及人工操作的效率。随着深度学习的广泛使用，可以基于海量的开源代码来进行漏洞识别的工作。

从第一种程序语言的发明到现在成百上千种语言的风靡，对于程序漏洞的分析一直是所有公司和开发人员的需求。当前已有许多在静态分析和动态分析领域的优秀的代码分析器，它们可以识别软件中的常见漏洞。Clang^[48]中的静态分析器，无需执行程序即可完成代码分析。而动态分析器的运作方式，则是在处理器上重复执行包含多种测试输入的程序，如果在程序执行过程出现不符合预期的输出，则表明程序有漏洞存在。静态分析和动态分析的共同点是，它们都是基于规则的工具，因此它们的效果依赖于规则设计的水准，同时也受限于规则，而且无法覆盖所有可能的程序路径。King^[49]使用符号值代替输入数据，并分析它们在程序的控制流程图中的使用情况，虽然做到可以覆盖所有可行的程序路径，但是符号执行的代价过于高昂，导致无法顺利扩展到大型程序。

2.2 自然语言处理

在自然语言处理中，对于人类创造的软件代码，基于这样一个假设^[50]：软件是人类交流的一种形式，软件的语料库和自然语言的语料库都是类似的，可以利用这些属性来构建更好的软件工具。在基于 NLP 的方法中，输入代码通常被处理成令牌，然后嵌入到连续的低维空间中。

Chris 等人^[21]开发了一个深度神经网络，可以构建代码的适当表示，并学习如何最优化这种表示，从而避免了对于专家经验的依赖，他们的模型在大多数任务中与手工构建的特征相比水平相当甚至更好。Miltiadis 等人^[51]提出了一种用于处理程序修复问题变体的架构，名为 SmartPaste，并以数据流敏感的方式来学习变量的位置和变量的上下文信息。Uri 等人^[23]通过将代码分解为抽象语法树中的路径集合，从而将代码片段表示为单个固定长度的代码向量。Eunjung 等人^[52]使用基于图的代码特征表示，使用程序的中间表示和机器学习算法来预测良好的优化序列。Tomas 等人^[53]对 Skip-gram 模型提出了改进，通过对频繁出现单词的二次采样，可以学习更高质量的向量表示。

2.3 循环神经网络

在传统的神经网络模型中，包含输入层、隐藏层和输出层，如图 2-1 所示，层与层之间是全连接的，而层内的节点之间是无连接的。这种传统形式的神经网络对于与时间序列有关的问题是无能为力的。典型的应用是语言模型，因为一个句子中的单词之间有着紧密的联系，当需要预测句子中下一个单词时，需要前面若干单词的信息，而该模型只能利用当前单词来预测下一个单词。

循环神经网络是为序列模型而生的^[54]，之所以被称为循环神经网络，因为其“循环”使用“过去”的信息。循环神经网络会对前面的信息进行记忆，并应用于当前输出的计算中。与传统神经网络不同的是，其隐藏层节点之间是有连接的，并且隐藏层的输入不仅包括输入层的输出，还包括上一时刻隐藏层的输出^[55]。在实现中，循环神经网络通过维护一个隐藏激活单元的向量来记录历史信息，这个隐藏状态可

以作为到目前为止在序列中出现的所有内容的表示，如图 2-2 所示， X_t 为 t 时刻的输入， O_t 为 t 时刻的输出，而 S_t 为 t 时刻的状态，该状态接收了 $t-1$ 时刻的状态 W 。循环神经网络理论上可以处理任意长度的序列数据，但是在实践中，为了降低复杂性，一般只假设当前状态只与前面几个状态相关。

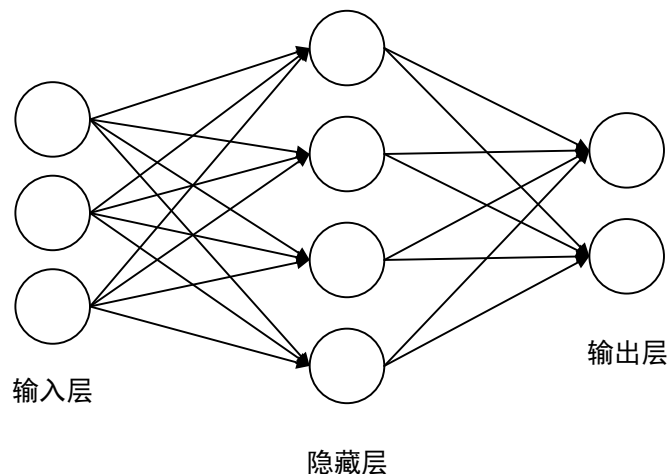


图 2-1 神经网络

由于在语言模型和机器翻译上的优秀表现，循环网络受到了极大的欢迎。然而，伴随着循环网络的不仅是极具吸引力的理论性能，还有在实践中难以训练的缺陷^[56, 57]。在实际应用中更常见到的，则是为了提高性能而设计的更复杂的循环结构，例如 LSTM^[58]和 GRU^[59]。

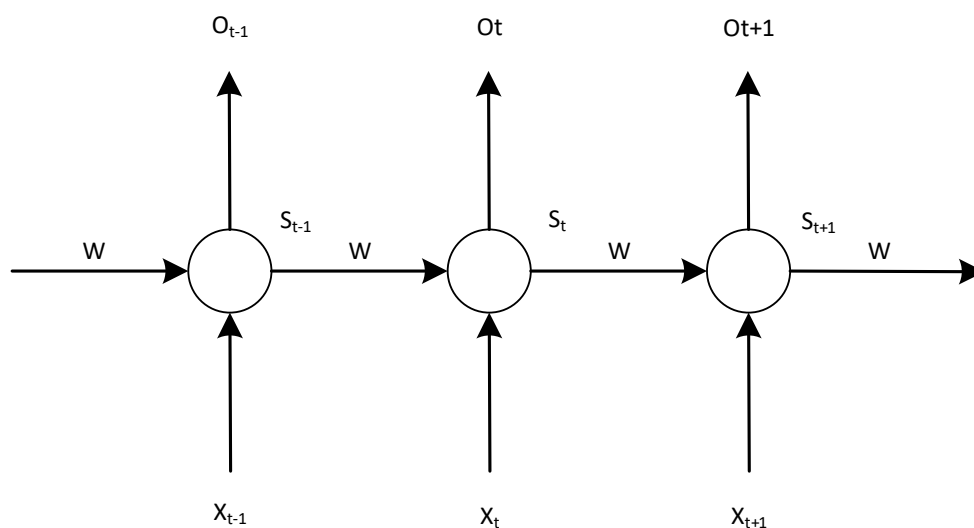


图 2-2 循环神经网络

2.4 卷积神经网络

在深度学习领域，序列模型通常被认为是循环网络的专业。包括 Goodfellow^[60]和吴恩达在内的行业领军人物都有将循环网络应用于序列任务的相关课程。然而最近的一些研究让科研人员把目光也转向了卷积网络。包括 van den Oord 等人^[17]在音频合成方面的应用，Kalchbrenner 等人^[15]在单词级语言模型方面的应用，以及 Gehring 等人^[16]在机器翻译上的应用等，都达到了当前顶尖的水平。这让人们重新思考卷积网络在序列任务中的应用前景。

不变性对于神经网络的训练是十分重要的，它使得预测模型趋向于满足特定任务的期望属性。例如卷积神经网络中的池化层可以捕获移位信息的不变性。之所以用卷积神经网络来处理图像，是因为进行卷积处理后，不同的图像有着一些共通的属性，即在一个图像区域的特征极有可能在另一个区域同样适用。所以在描述一个较大的图像时就可以对不同位置的特征进行聚合统计。这种处理方式不仅可以降低图像的规模，还不容易过拟合。在处理一维序列时，同样要求模型捕获全局的不变性特征，而不是单个样本或单个区域的独特性。

2.5 本章小结

本章介绍了传统代码漏洞检测和机器学习代码漏洞检测的相关技术。传统方法包含静态分析和动态分析两类，它们都是基于规则的工具，有其自身的局限性。而对于程序代码的处理，通常借鉴了自然语言处理的相关理论与研究。其中包含了基于循环神经网络和卷积神经网络的模型，它们都可以用于代码的处理与分析，也有着各自的优缺点。

3 基于深度学习的代码漏洞检测模型

3.1 模型总体架构

代码漏洞检测模型的总体架构如图 3-1 所示。整个系统按照数据的处理阶段自底向上分为三个部分：数据编码预处理模块、数据加载模块、时序卷积神经网络模块。

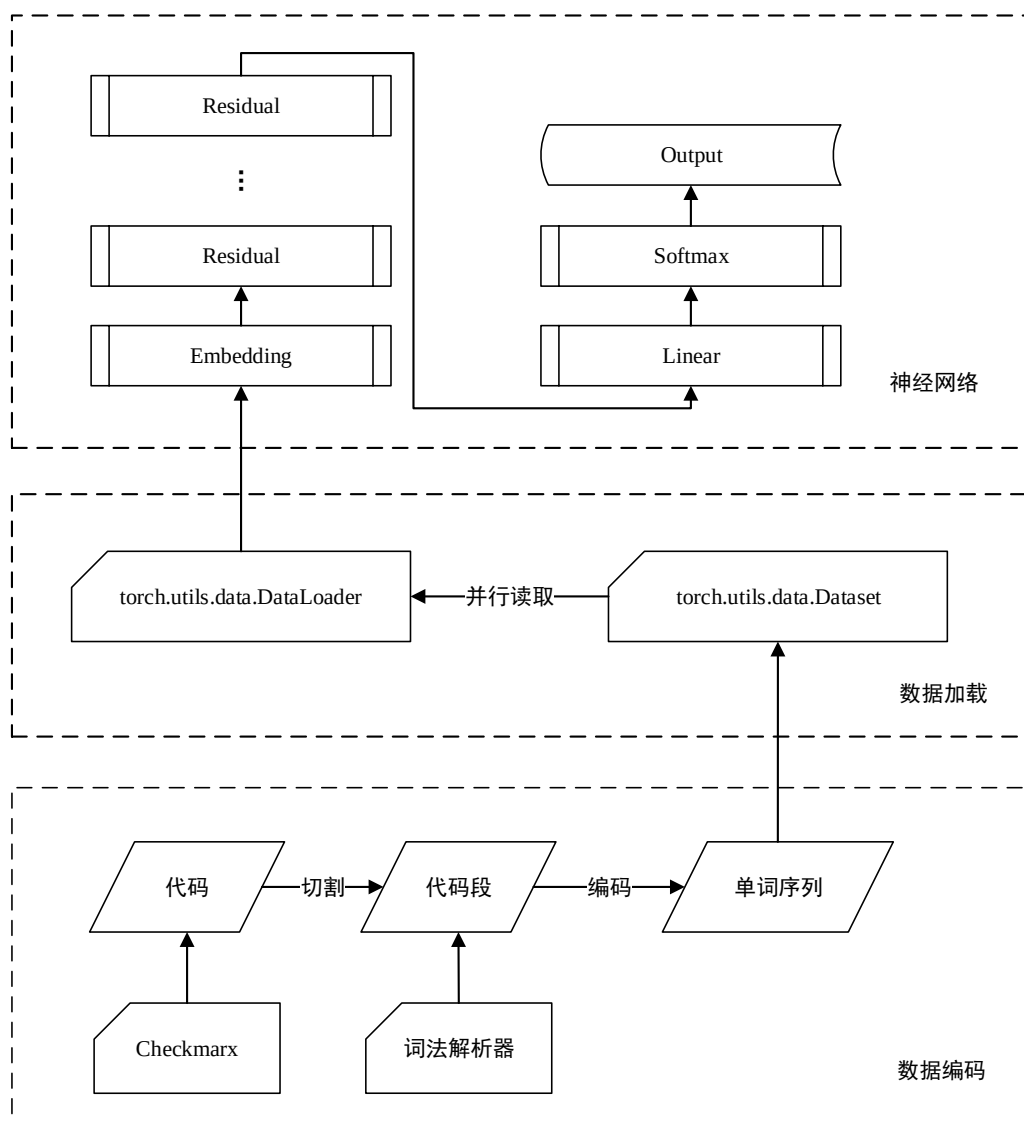


图 3-1 模型架构图

数据编码预处理模块负责将原始的函数代码编码为单词序列，生成适用于神经网络输入的数据文件。这其中包含了使用软件将完整代码切割为代码段，以及使用词法分析器将代码段编码为单词序列两个过程，具体的操作流程将在 4.1 节描述。

数据加载模块负责读取编码预处理模块生成的单词序列文件，并分批输入到神经网络中。首先使用 `Dataset` 类将数据统一到固定长度并规整到神经网络可接受的维度，然后使用 `DataLoader` 类将数据分批送入神经网络。通过使用 `PyTorch` 的数据加载接口，这个过程是可并行化的，可以使用多线程进行加速。在 4.2.1 节中将详细描述如何使用 `PyTorch` 的数据加载 API 来加载自定义的数据文件。

时序卷积神经网络负责接收数据加载模块的送达数据，并通过训练数据优化自身的网络参数，在测试阶段经过计算得出每个样本的正负概率，以判断样本对应的代码段是否包含漏洞。网络首先将输入数据经过词嵌入层（`Embedding`），该层负责将数值编码为低维词向量，然后数据通过多层残差单元（`Residual`），对数据特征进行提取，最终线性层（`Linear`）将输出数据的维度限制在 2 维，并做 `softmax` 得到每个样本的正负预测概率。

3.2 代码编码预处理

代码编码预处理方案包含两个阶段的过程：代码特征提取和代码序列编码。其中代码特征提取是为了缩减代码语句的规模，保留关键的信息，过滤与漏洞无关的代码，以防止冗余信息对于代码漏洞的检测。代码序列编码是为了将代码转换为神经网络可接受的数值型输入。代码编码预处理方案设计如图 3-2 所示，首先用软件将函数切割为代码段，该过程仅保留了分析漏洞所需的关键代码语句，去除了非相关语句。然后使用词法分析器对代码段做词法分析，按照代码语句顺序代码数据，得到单词向量的字典，然后再对代码数据进行第二轮遍历，该过程中对单词序列进行编码，得到单词向量序列。

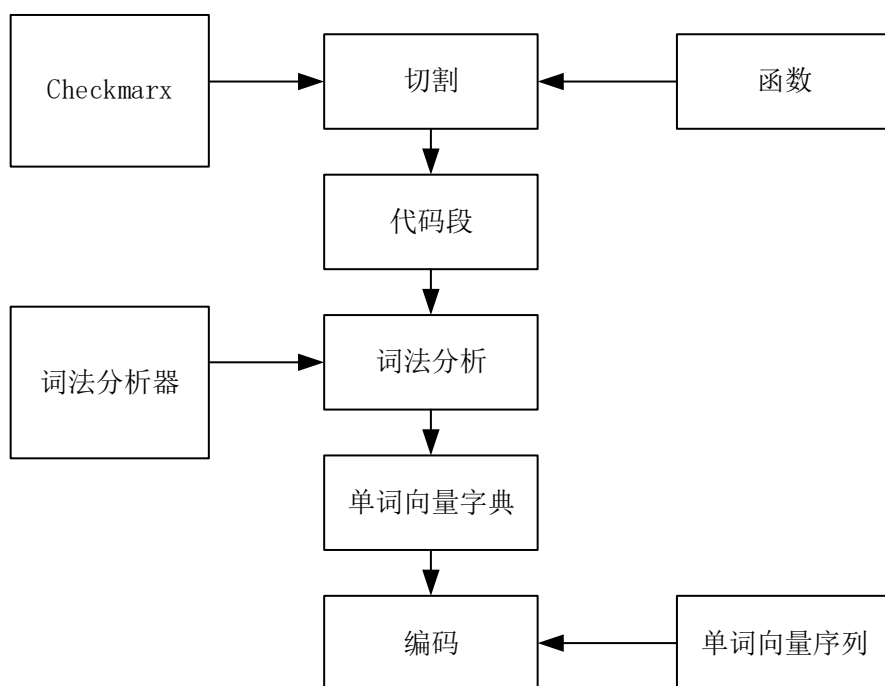


图 3-2 代码编码处理

3.2.1 代码特征提取

为了解决 1.3 节中的第一个问题，使得代码预处理既能够保留足够的信息来判断漏洞，又可以尽量减少冗余信息，参考 Li 等人^[44]的处理方式，他们在文中提出了一项程序代码转换为向量的指导原则：程序代码应该首先转换为一种中间表示，该中间表示能够保留程序元素之间的语义关系，例如数据依赖性和控制依赖性，然后再将中间表示转换为向量，作为神经网络的原始输入。其表述符合代码漏洞检测模型对于代码预处理的要求，之所以有这样的要求，首先因为神经网络仅接受向量输入，必须将程序转换为向量；同时转换为向量后必须要保留程序的语义信息。

为了将程序表示为有语义的向量，先将程序表示为一些小型的代码段，每个代码段包含许多程序语句。语句在空间上不一定是连续的，但是在数据依赖性 or 控制依赖性上彼此相关。为了让代码段能包含漏洞所在的位置，通过引入关键点概念来定位有漏洞的代码段。所谓关键点是可能造成漏洞的原因所在，而关键点和漏洞类型之间的关系是多对多的，例如缓冲区错误漏洞类型可能是由 API 调用、数组的使用、指针的使用造成的，则这些关键的代码位置都是缓冲区错误漏洞类型的关键

点。同样，一种关键点也可能造成多种漏洞类型，例如 API 调用关键点可造成缓冲区错误和资源管理类漏洞。Li 等人在处理关键点的问题上，专注于 API 的调用，而此类型的代码段可由数据流图或控制流图来生成，并且有商业软件可以使用。

代码特征提取过程如图 3-3 所示，首先将软件代码中的函数和 API 调用提取出来，得到一个个独立的函数代码，然后提取函数中的代码段，这些代码段中就包含了可定位漏洞的关键点，最后为每个代码段做标记，标明该代码段是否含有漏洞。

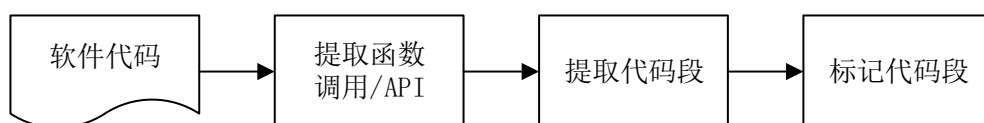


图 3-3 代码特征提取

3.2.2 代码序列编码

在将代码段进一步转换为向量的过程中，没有沿袭 Li 等人的方案，而是采用了简洁直观的词法分析法，即用词法分析器对代码段进行分析，然后做向量编码。代码序列编码过程如图 3-4 所示，通过使用词法分析器对所有代码段进行分析，词法分析器会将代码中的关键字、变量和常量提取为一个个的令牌。然后对令牌进行过滤，去除名字重复的令牌，即得到一个没有重复内容的词汇表。进而对词汇表中的单词依次编码为数值，最终将数值编码为多维向量，该向量即为对应令牌的多维表示。经过整个处理过程，代码段数据（单词序列）被转换为向量序列。

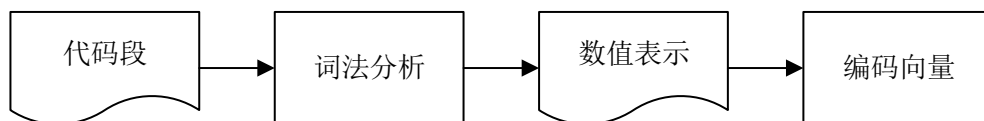


图 3-4 代码序列编码

在对令牌进行过滤时，在所有同名的令牌中，仅保留了一个名字。对于语言的关键字和常量来说，这种过滤操作是恰当的，因为在所有代码段中出现的同名的关键字和常量本质上是同一种语义。例如 C 语言中的 static 关键字，在所有出现 static 关键字的地方，该符号都表示 C 语言中的这一关键字，而不会是一个变量名或一个函数名，毕竟用户命名和语言关键字冲突的行为在语法上是不允许的。而对于变量和函数名来说，这种过滤操作可能不甚合适，因为在不同地方同一个名字可能表达

不同的语义。例如同样被命名为 `count`，在某代码段里可能表示一个 `int` 型变量，在另一个代码段里可能表示一个函数名，这就会导致所有同名的用户命名会被当成同一种语义。尽管这种处理方式会导致全局上的语义不准确，但是神经网络的样本对象是单个代码段，而在一个代码段中，并不会出现同名的用户命名表示不同的语义。所以这种全局性的语义不准确丝毫不会影响神经网络对代码漏洞的分析。

3.3 时序卷积神经网络

在介绍卷积网络结构的设计之前，首先必须强调的是，程序代码的分析工作，本质上是一个时间序列模型。对于一个给定的输入序列 $x_0, x_1, \dots, x_T$ ，其中下标表示时间，基于该输入序列，模型可以预测相应的输出序列 $y_0, y_1, \dots, y_T$ 。根据时间序列的特点，当预测 t 时刻的输出 y_t 时，只能利用 t 时刻之前（包括 t 时刻）的输入： $x_0, x_1, \dots, x_t$ 。用公式表示，一个序列模型就是一个函数 $f: X^{T+1} \rightarrow Y^{T+1}$ ：

$$\hat{y}_0, \hat{y}_1, \dots, \hat{y}_T = f(x_0, x_1, \dots, x_T)$$

其中在预测 y_t 时只依赖于 $x_0, \dots, x_t$ ，而不依赖于任何未来的信息 $x_{t+1}, \dots, x_T$ 。序列模型学习的目标就是找到最优的网络 f ，使得实际输出和预测值之间的损失函数 $L(y_0, y_1, \dots, y_T, f(x_0, x_1, \dots, x_T))$ 取得最小值。

序列模型适合于许多任务类型，比如自回归预测，通过将目标输出设为输入右移一个时间步，从而利用过去的信息来预测未来。但是它并不适用于机器翻译或 `sequence-to-sequence` 任务，因为在这些任务中整个输入序列（包括未来的信息）都用来预测当前输出。

除时序问题之外，模型能够采集的序列长度也是一项重要的指标。由于循环神经网络可以处理长期依赖问题，所以卷积网络也要获得类似的能力才能处理同类问题。在神经网络中，感受野的概念定义了网络内部不同位置的神经元对于原始序列的感受范围。由于卷积网络使用卷积层和池化层，而层与层之间都采用局部相联，所以单个神经元无法感知到原始序列的所有信息。神经元感受野越大，表示它接触到的原始序列范围越大，于是它能够表达更为全局或语义层次更高的特征；而神经

元感受野越小，它更趋向于表达局部特征和细节信息。由于本文模型对于代码的检测是函数级别的，所以理论上模型的感受野需要覆盖绝大多数的函数长度，即每个函数所包含的词汇数量。

3.3.1 因果卷积

为了解决 1.3 节中第二个问题，以满足模型对于时序问题处理上的要求，在每一步卷积时丢弃 t 时刻之后的数值，从而使 t 时刻的输出只依赖于 t 时刻之前的输入。这种处理方式被 van den Oord 等人^[17]提出，用于音频数据的生成，并被命名为因果卷积。在神经网络中，过滤器（或称为卷积核）的作用是去除输入中的部分信息，仅留下有效的信息。过滤器在每个时间节点，输出由输入的加权决定，这项操作称为卷积。当卷积不依赖于未来的信息时，即为因果卷积。

图 3-5 为一个 4 层的因果卷积网络示意图，网络共 4 层，首层为输入层，中间两层为隐藏层，最后一层为输出层。每个单元格表示卷积层中的一个单元，单元之间的箭头表示数据的正向计算。图中完整的画出了和 y_T 相关的所有单元以及数据传播路径，并将数据传播箭头加粗表示，其他部分则只描绘了部分，以保证整个图的整洁。可以看到，当计算 y_T 时，仅用到了 x_T 及之前的输入，没有用到之后的输入，这满足因果卷积的要求。

由于代码序列是一维数据，所以使用了一维全连接网络，并且为了保证每个隐藏层和输入层长度相同，使用了长度为 $k-1$ 的零填充，其中 k 为卷积核大小。

与循环网络相比，使用卷积网络进行序列数据建模的优势在于计算的可并行化。循环网络中，后面时间步的预测必须等待其前任完成，而卷积运算由于在每层使用相同的卷积核，从而可以并行完成卷积。因此，在训练和预测中，卷积网络可以将整个输入序列同时处理，而不是像循环网络中依次处理。而且卷积网络的内存需求更低，在处理长输入序列时，循环网络需要消耗大量的内存来存储其多个单元的部分结果。但是在卷积网络中，一个层中共享本层所有卷积核，反向传播路径长度仅取决于网络深度，没有额外的信息存储需求。因此在实践中，卷积网络的内存消耗远小于循环网络。

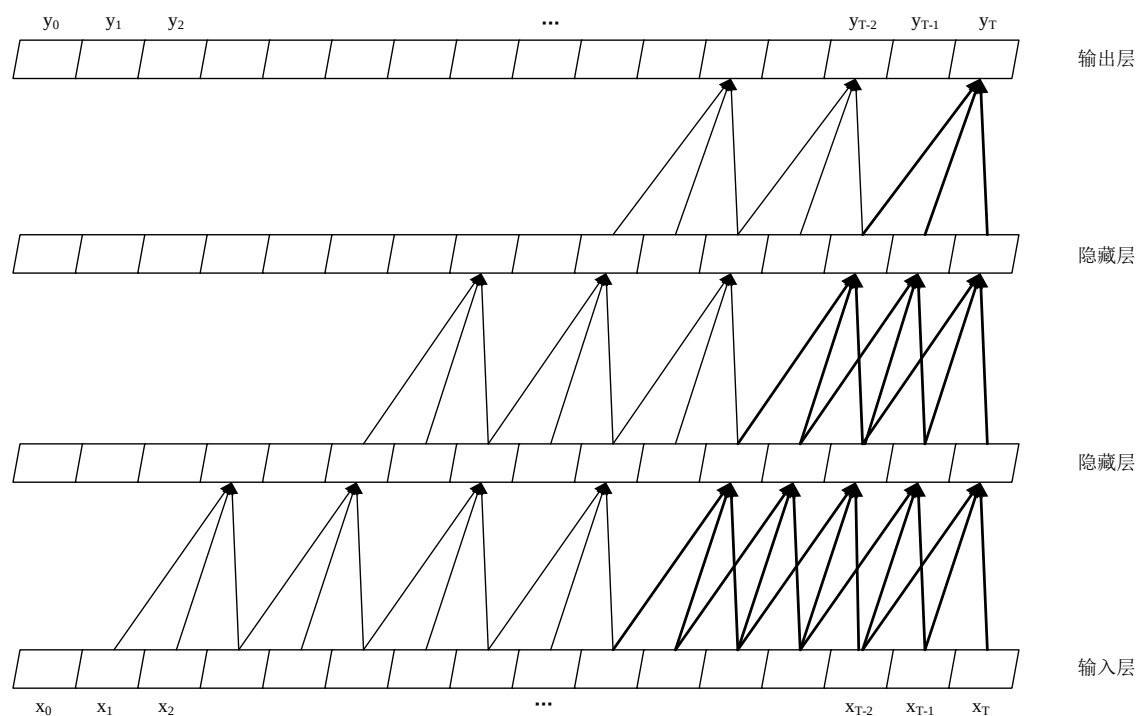


图 3-5 因果卷积

有了因果卷积和全连接网络后，网络模型已经满足时序性要求，然而想要处理长序列，并建立长远有效的历史数据，则需要极深的网络结构和非常大的卷积核，才能有足够大的感受野来覆盖每一个代码段样本。从图 3-5 中很容易看到，随着网络层数的增加，感受野尺寸只有线性的增长，这远不能满足处理长序列的需求。因此，接下来将结合现代卷积网络中的技术，解决网络的数据长期依赖问题。

3.3.2 扩张卷积

为了解决 1.3 节中的第三个问题，使得模型有处理长期依赖问题的能力，在网络中加入扩张卷积技术。由于简单的因果卷积只能保留网络深度线性大小的历史信息，这使得因果卷积很难直接应用于序列任务，特别是对于代码分析这类需要长期历史信息的任务。使用扩张卷积可以使每一层神经元的感受野随着网络深度呈指数型增加。扩张操作通过在卷积核的抽头之间引入固定步长，从而跳过部分输入，使得卷积核可以采样到大于其尺寸的区域信息。当步长为 1 的时候，扩张卷积就退化

为普通卷积。使用更大的扩张系数可以使最终输出能够包含更广泛的输入信息，从而扩展了网络的感受野。

对于一个 1 维序列 $x \in R^n$ ，卷积核为 $f: \{0, \dots, k-1\} \rightarrow R$ ，扩张卷积操作 F 在序列中某元素 s 上的运算为：

$$F(s) = (x * _d f)(s) = \sum_{i=0}^{k-1} f(i) \cdot x_{s-d \cdot i}$$

其中 d 为扩张系数， k 为卷积核尺寸。

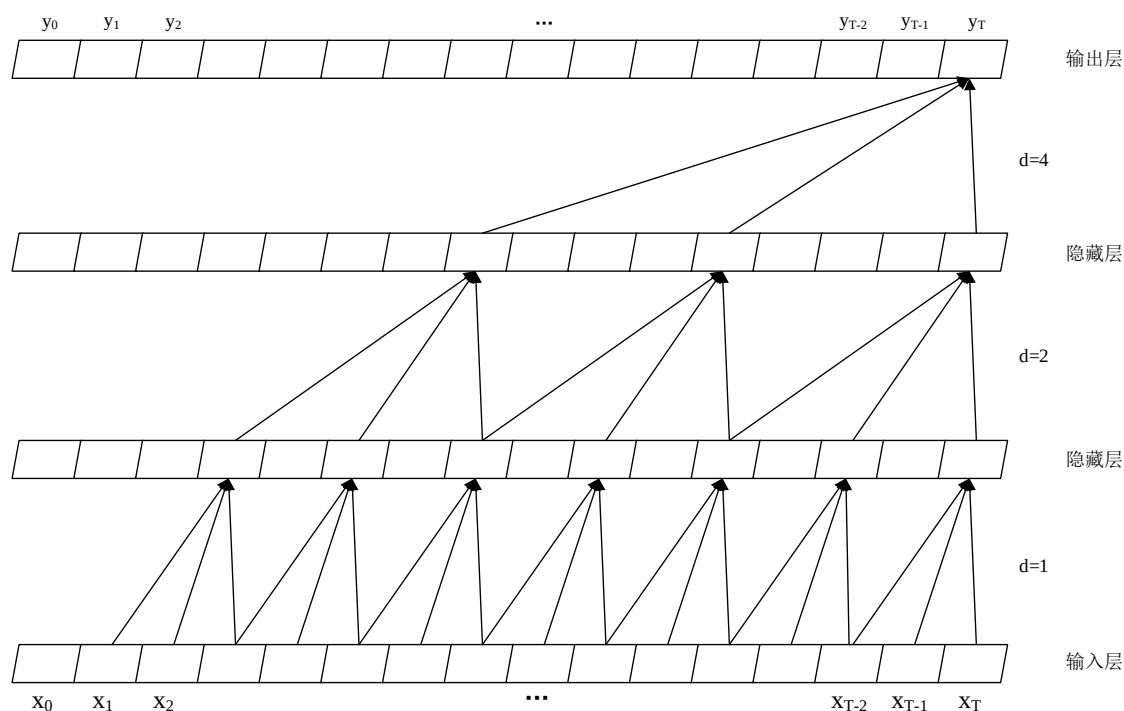


图 3-6 扩张卷积

图 3-6 是一个扩张系数指数增长的卷积网络示意图，网络共 4 层，首层为输入层，中间两层为隐藏层，最后一层为输出层。每个单元格表示卷积层中的一个单元，单元之间的箭头表示数据的正向计算。图中并没有将每一个单元的数据传播都描绘出来，因为这样会影响整个图的观感和表达的清晰度。其中每层之间的扩张系数分别为 1、2、4，当系数为 1 时，可以看作是普通的卷积层。

引入扩张卷积后,现在有多种方法可以增大网络的感受野:更大的卷积核尺寸、更大的扩张系数或更深的网络层数。通过调整参数,可以更好的控制模型的内存消耗,并且使模型通过调整参数来灵活适应不同领域的任务。当使用扩张卷积时,让扩张系数随网络深度指数增长,网络随之获得极大的感受野。这确保输入的每一项都被卷积核选中,并且随着网络深度的增加可以获得极大的有效历史数据。

3.3.3 稳定梯度

网络模型的感受野依赖于网络深度、卷积核尺寸以及扩张系数,当模型的深度增加时,训练的稳定性就变得尤为重要。例如,当预测值依赖于 2^{12} 长度的高维历史数据时,网络可能需要多达 12 层卷积,对于每一层来说,又有多个卷积核来收集特征。为了稳定训练过程,在网络中使用残差单元,而不是普通卷积单元。

神经网络基于反向传播算法来计算梯度,由于在计算隐藏层梯度时用到了链式法则,梯度值会进行一系列的连乘,导致层数太深时隐藏层的梯度会出现严重的衰减,即梯度消失问题。通过使用 ReLU 代替 Sigmoid 激活函数,可以在一定程度上抑制梯度爆炸和梯度消失问题,这是由于 ReLU 函数使网络自行引入了稀疏性,降低了神经网络的饱和度,同时大大提高了训练速度,并降低计算成本。即使是这样,在网络深度到达一定层级时,ReLU 也无法完全避免成百上千次的连乘所造成的梯度消失。

He 等人^[61]提出的残差网络即是为了优化深层卷积网络的训练。假设多个非线性层的组合为函数 F ,则残差单元的输出由多个卷积层级联的结果和原始输入相加,再经 ReLU 激活得到,其中还要保证卷积层输出和输入元素维度相同,用公式表达残差单元的输出为:

$$O = \text{Activation}(x + F(x))$$

单个残差单元的结构如图 3-7 所示。在残差单元中,有两层扩张因果卷积和 ReLU 单元,并且对卷积核采用权重正规化。另外,在每个扩张卷积采用空间 dropout 来做正则化,即在每个训练步之后,整个通道归零。在标准的 ResNet 中,输入直接与残差函数的结果相加,然而在代码漏洞检测模型中,输入和输出可能有不同的

长度。为了解决这种差异，网络使用额外的 1×1 卷积来确保向量加法的输入有相同的维度。

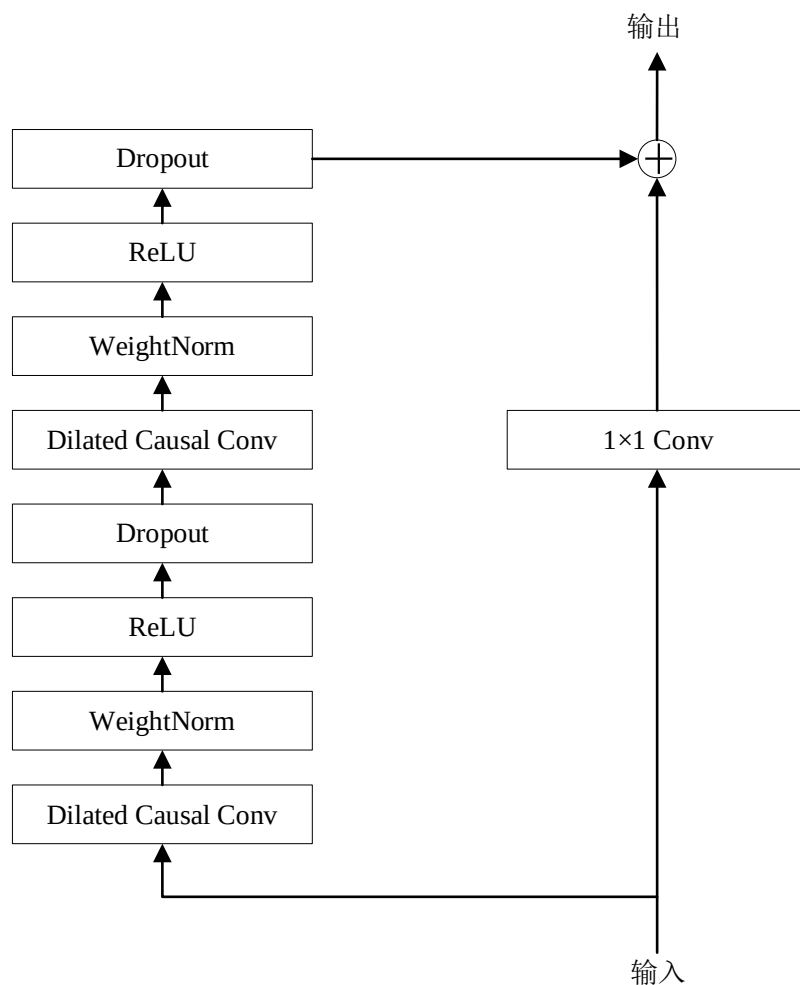


图 3-7 残差单元

通过使用残差网络，可以更好地控制参数数量，保证网络规模不会过于庞大。而且使用了更少的池化层，更多地使用下采样，提高了反向传播的速度。网络中存在明显的层级，随着网络深度的增加，每层的特征图数量逐渐增加，特征表达能力也随之增强，而且当层数较高时，还可以缩小卷积核的尺寸，从而控制输出特征图的数量。

3.4 时序卷积神经网络调参准则

在 3.3.2 节中重点研究了扩张卷积对于卷积神经网络的感受野的影响，而感受野的大小对于卷积神经网络性能的影响是关键性的。神经网络的表达能力受到网络规模的限制，对于卷积神经网络来说，增加网络层数（深度）和增加每层的卷积核数或增大卷积核尺寸（宽度），都可以增强其表达能力。在相同时间复杂度的前提下，如何平衡神经网络的深度和宽度，以取得网络表达能力的最大化，He 等人^[62]进行了大量实验并进行了定量的分析。其结论是在时间复杂度相同时，较深的网络表达能力更强。然而他们同时指出，网络也并非越深越好。除去计算资源的消耗问题，在网络深度达到一定量时，继续增加层数并不能提高性能，仿佛网络深度对于表达能力的提升存在阈值。更糟糕的是，如果继续增加网络的层数，网络性能反而会出现退化。而且由于训练误差也随着深度的增加而提升，所以不能将原因归咎于参数增加造成的过拟合。

所以对于特定的数据集，神经网络的参数规模并不是越大越好。除了可能会出现典型的过拟合问题外，还会有其他目前未研究透彻的问题出现，而且过大的网络对于计算资源的需求也是巨大的，这会带来昂贵的设备成本。对于处理时序数据的卷积神经网络来说，使得网络的感受野接收到足够长度的序列信息是必要的。这就对神经网络的参数规模有了进一步的约束。

为了使神经网络的感受野足够覆盖序列数据长度，将可能的参数及对应的感受野列于表 3-1 中。当代码序列数据的长度为定在 200 时，可以看到当网络层数或卷积核尺寸过小时，感受野是远远不够的。然而，当感受野和序列长度相近时，感受野大小是否一定要大于序列长度才能得到合格的结果？当感受野大于序列长度时，是否感受野越大准确率就越高？如果是，提升程度又有多少？这些问题经过大量的实验验证都得到了解答。

首先，当神经网络的感受野远小于序列长度时，网络完全没有能力收集和处理序列数据，在实验中体现为：训练集损失和测试集损失都居高不下，准确率相关指标也极差。然后，当感受野与序列长度相近时，网络的表达能力会有质的飞跃，在

实验中体现为：当网络深度再增加一层或卷积核尺寸再增加一个步长时，而导致感受野达到与序列长度相似的水平，此时网络的进化使得经过一定轮次的训练后，训练集损失和测试集损失可以降低到合理的数值，并且准确率相关指标也会有巨大的进步。最后，当感受野大小足够时，通过更深的网络或更大的卷积核来获得更大的感受野，对于最终的结果没有明显的帮助，甚至如果强行使用规模明显过大的网络，还会导致过拟合的提前出现以及训练指标的波动。

表 3-1 感受野尺寸

卷积核 网络层数	3	5	7	9
1	3	5	7	9
2	7	13	19	25
3	15	29	43	57
4	31	61	91	121
5	63	125	187	249
6	127	253	379	505
7	255	509	763	1017
8	511	1021	1531	2041

综合而言，对于特定数据集来说，为调整时序卷积神经网络的超参数适配于该数据集，应该使得网络的感受野与序列长度相当。过小的感受野会使得网络表达能力不足，而过大的感受野不仅使网络规模过于庞大而耗费大量计算资源，甚至并不会使最终结果有进一步的提升。

3.5 本章小结

本章描述了基于深度学习的代码漏洞检测模型的设计。首先介绍了模型的总体架构。然后说明了对代码类型的数据预处理方法，先将代码提取为包含语义信息的中间表示，而后对代码段进行词法分析并编码为向量。接着描述了用于代码漏洞检

测的时序卷积神经网络，先用因果卷积构建一个时间序列模型，再用扩张卷积在避免网络过深的前提下增大感受野，最后用残差单元稳定梯度并加快计算速度。最后，介绍了面对特定的数据集时，时序卷积神经网络的调参准则。

4 代码漏洞检测模型的实现

4.1 代码编码预处理

Li 等人构建了一个用于评估基于深度学习的代码漏洞检测系统性能的数据集，并提供了原始代码以及使用 Checkmarx 生成的代码段，由于 Checkmarx 的处理有小概率会产生错误，导致程序语句不完整，这会使得严格的词法分析程序报错。所以，首先对代码段的数据进行人工筛查，删除少量存在残缺语句的样本，然后进行词法分析及编码。

由于单次处理数据的时间较长，为了实现部分中间结果的复用，以减少处理过程中出错的成本，将整个代码编码过程分为了两个阶段，如图 4-2 所示。首先对代码段进行第一遍词法分析，记录所有出现过的单词，并对单词顺序编码。代码数据的样式是一个个的代码段，每个代码段标注了其来源和标签（0/1），单个代码段样例如图 4-1 所示，首行第一个数字是样本编号，接着是代码样本的源文件及函数名，从第二行开始是代码段本体，直到倒数第二行，而最后一行即为标签，0 表示样本无漏洞，1 表示样本存在至少一种漏洞。然后将单词及其编码以字典的形式进行存储，其中单词为键，编码为值，这样在第二阶段处理时就可以方便地以单词查找其对应的编码，最后将字典保存在中间结果文件中。

```
1 CVE-2010-1444/vlc_media_player_1.1.0-CVE-2010-1444_zipstream.c cfunc 449
ZIP_FILENAME_LEN, NULL, 0, NULL, 0 )
char *psz_fileName = calloc( ZIP_FILENAME_LEN, 1 );
if( unzGetCurrentFileInfo( file, p_fileInfo, psz_fileName,
vlc_array_append( p_filenames, strdup( psz_fileName ) );
free( psz_fileName );
0
```

图 4-1 代码段样例

第二阶段流程如图 4-2(b)所示，需要同时读取源代码文件及中间结果文件，然后是类似第一阶段的遍历过程，不过每次获取 token 后就将其编码添加到序列中，

每个代码段处理结束后会加上其标签，标签同样是在源文件中读取的。最终输出的数据文件即是代码的编码序列及其标签，可用于神经网络的输入。需要指出的是，在神经网络的首层添加了嵌入层，可以在网络的计算过程中实现词嵌入编码，而不必在数据处理过程中进行编码。与在预处理时就进行编码相比，其优势在于数据文件的大小得到了有效的控制，而且神经网络的训练过程可以用 GPU 进行加速，与用 CPU 做词嵌入编码相比运算速度也大大加快。

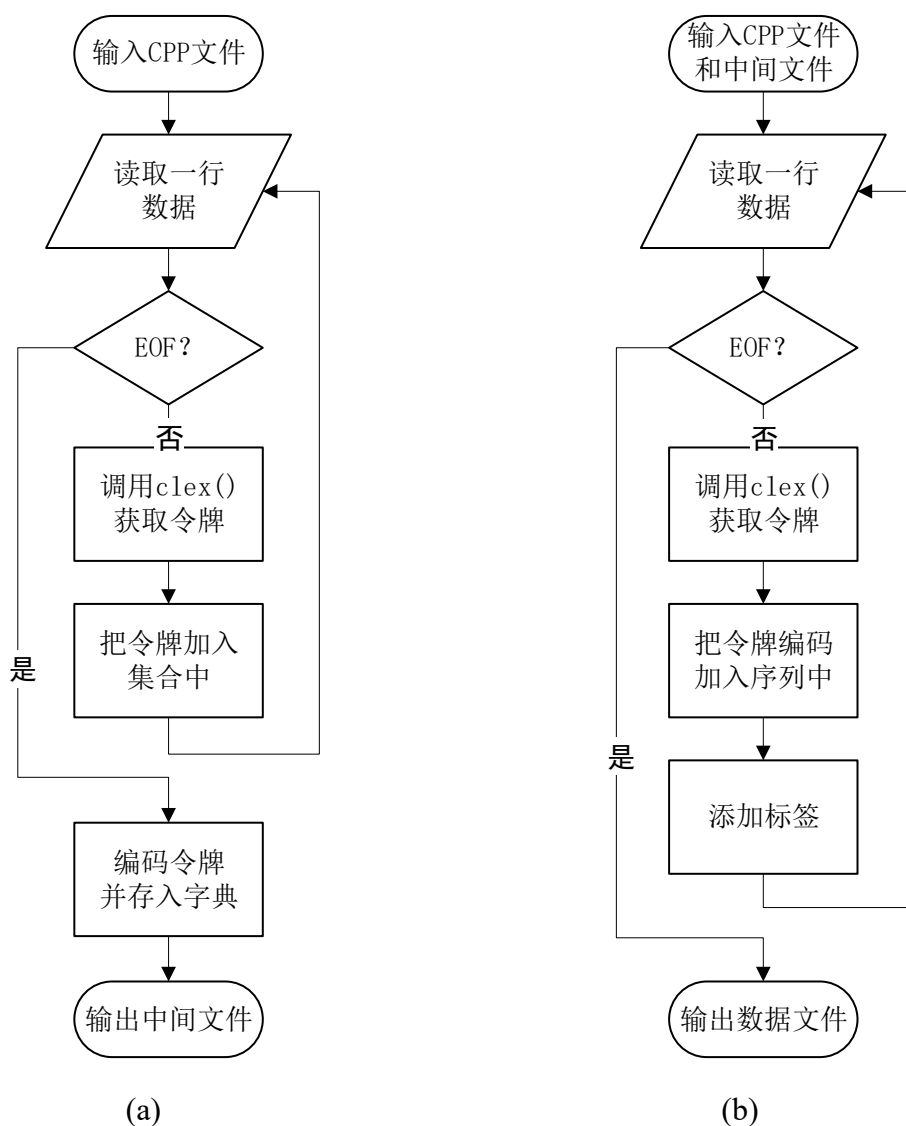


图 4-2 代码编码流程

4.2 时序卷积神经网络

4.2.1 数据加载器

在数据加载的过程中，使用了 PyTorch 中的数据加载接口，具体流程如图 4-3 所示。首先 CWELoader 类从文件中读取已经预处理过的数据，然后进行顺序打乱和分割。这里按照 8: 2 的比例来分割训练数据和测试数据，而顺序打乱操作是为了使每次训练时的训练数据和测试数据都是随机取出的，保证两种数据在数据集中的均匀性。得到训练数据和测试数据之后，trainLoader 类和 testLoader 类分别对两部分数据对齐到同样的长度。经过分析所有样本的长度分布后，序列长度被统一在 200，此时可覆盖大部分的样本而不必切割尾部，而不足 200 长度的序列用 0 补足即可。最后使用 PyTorch 中 utils.data.DataLoader 类对数据进行打乱顺序，并以特定的 batch 大小使用多线程的方式高效地取出。

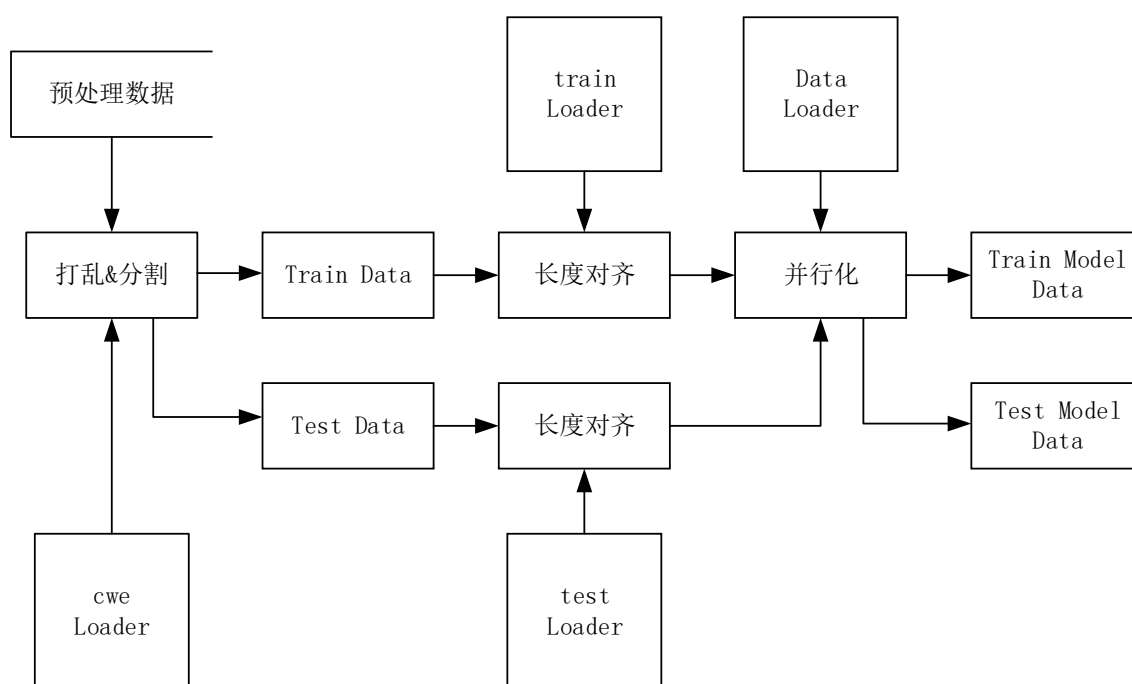


图 4-3 模型数据加载

4.2.2 网络模型构建

在第三章已经详细说明了在神经网络模型中所用到的技术。在实现时，使用了当前在科研领域非常流行的深度学习框架 PyTorch。PyTorch 最具吸引力的优势在于其图结构是动态的，图在模型运行时才进行构建，这使用户可以像调试 Python 代码一样调试 PyTorch，并且可以方便地使用循环。与 TensorFlow 等使用静态图结构的框架相比，这样的调试方式显然更直观并且方便。而且 PyTorch 可以视为 Numpy 的增强版，对于熟悉 Python 语言的研究者来说，学习成本相当低。最后，PyTorch 有着完善的数据加载接口，接口由数据集、取样器、数据加载器组成，而且并行化数据加载也十分简单，这对于需要将预处理的自定义数据集输入到模型的研究者来说无疑是非常方便的。

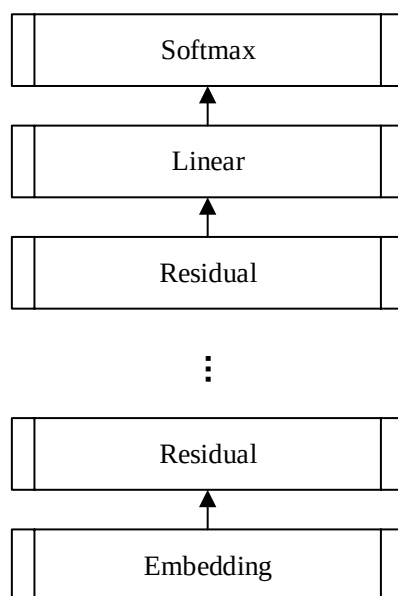


图 4-4 代码漏洞检测网络

整个网络的设计如所图 4-4 示，其中 Residual 表示单层的残差单元，其结构在第三章已经说明，现将残差单元的实现代码列于图 4-5 中。可以看到残差单元的代码实现符合图 3-7 的设计，单层残差网络包含两层因果卷积，因果卷积的实现即为普通卷积附加一个 Chomp1D，Chomp1D 切割掉 t 时刻之后的输入，从而达到因果卷积的设计目标。每层因果卷积还包含一个 ReLU 激活函数以及 Dropout。由于采

用了残差单元，为了保证残差单元最后的向量加法的两个操作数有相同的维度，采用 1×1 的卷积作用于每层的输入。而在 4.1 节描述代码编码处理时，已经说明将词嵌入编码加入神经网络中的优势，所以网络的首层便是 Embedding 层，它将数值编码为多维向量。接下来是多层的残差单元，具体的层数以及每层的单元数，可根据任务的不同而改变。对于代码检测工作而言，具体的网络参数将会在第五章描述。由于漏洞检测工作的本质是二分类（是/否），所以在网络的最后加入线性层，将输出转换到 2 维，然后对输出做 softmax，得到每个样本的正负概率。

```
class TemporalBlock(nn.Module):
    def __init__(self, n_inputs, n_outputs, kernel_size, stride, dilation,
                  padding, dropout=0.2):
        super(TemporalBlock, self).__init__()
        self.conv1 = weight_norm(nn.Conv1d(n_inputs, n_outputs, kernel_size,
                                             stride=stride, padding=padding, dilation=dilation))
        self.chomp1 = Chomp1d(padding)
        self.relu1 = nn.ReLU()
        self.dropout1 = nn.Dropout(dropout)

        self.conv2 = weight_norm(nn.Conv1d(n_outputs, n_outputs, kernel_size,
                                             stride=stride, padding=padding, dilation=dilation))
        self.chomp2 = Chomp1d(padding)
        self.relu2 = nn.ReLU()
        self.dropout2 = nn.Dropout(dropout)

        self.net = nn.Sequential(self.conv1, self.chomp1, self.relu1,
                                   self.dropout1, self.conv2, self.chomp2, self.relu2, self.dropout2)
        self.downsample = nn.Conv1d(n_inputs, n_outputs, 1) if
                                n_inputs != n_outputs else None
        self.relu = nn.ReLU()
        self.init_weights()

    def init_weights(self):
        self.conv1.weight.data.normal_(0, 0.01)
        self.conv2.weight.data.normal_(0, 0.01)
        if self.downsample is not None:
            self.downsample.weight.data.normal_(0, 0.01)
```

```
def forward(self, x):  
    out = self.net(x)  
    res = x if self.downsample is None else self.downsample(x)  
    return self.relu(out + res)
```

图 4-5 残差单元代码实现

4.3 本章小结

本章详述了代码异常检测模型的实现。首先是代码编码的实现，代码编码过程分为两个阶段，第一阶段记录出现过的单词，并对单词顺序编码；第二阶段构建单词编码的序列。

然后说明了使用 PyTorch 构建神经网络的过程，包括数据加载器和神经网络本身两个部分。数据加载器作为数据和模型之间的适配器，将数据输入到模型中。而神经网络模型除了残差单元，还包括开始的嵌入层和最后的线性单元及 softmax。

5 模型测试与结果分析

5.1 实验环境

实验的硬件及软件环境如表 5-1 所示。

表 5-1 实验环境

环境	配置
CPU	Intel(R) Core(TM) i7-7660 CPU @ 2.5GHz
内存	16 GB
硬盘	512 GB
操作系统	Windows 10 专业版
Python 版本	3.6.4
Pytorch 版本	1.0.0
Torchvision 版本	0.2.1

5.2 实验数据

表 5-2 实验数据集

数据集 \ 参数	样本数	正样本数	负样本数
CWE-119	38023	9957	28066
CWE-399	20628	6744	13884
Mixed	58651	16701	41950

实验数据采用 Li 等人已公开的数据集，其中代码数据已经过 Checkmarx 处理，而标签也经过人工审核。在此基础上，删除了部分过长的样本及有截断错误的样本，保证编码程序可以正常运行。其中包含两类数据：CWE-119 数据集和 CWE-399 数

数据集。CWE-119 是内存缓冲区漏洞，包含了 38023 条样本，其中正样本 9957 条，负样本 28066 条；CWE-399 是资源管理漏洞，包含了 20628 条样本，其中正样本 6744 条，负样本 13884 条。最后将两个数据集合并，组成混合数据集。混合数据集包含了多种数据的特征，但仍然采用正常和异常两种标签，理论上检测的难度更大。三类数据集的信息列于表 5-2 中。

5.3 实验结果及分析

实验部分首先对数据量最大的混合数据集进行实验，从小尺寸的卷积核开始尝试，在保持卷积核尺寸 ($k=3$) 不变的情况下，逐渐增加网络层数，并记录各项指标。

图 5-1 是卷积核尺寸 $k=3$ 时的训练集损失和测试集损失，横轴是训练轮数，共训练 20 轮，不同网络层数的数据由不同形状的折线表示。可以看到除了网络层数为 5 时两种损失都居高不下以外，其他情况下的损失没有太大差异，而且测试集损失在 13 轮之后反而有略微增加的趋势，说明训练轮数太大时可能有过拟合的问题。

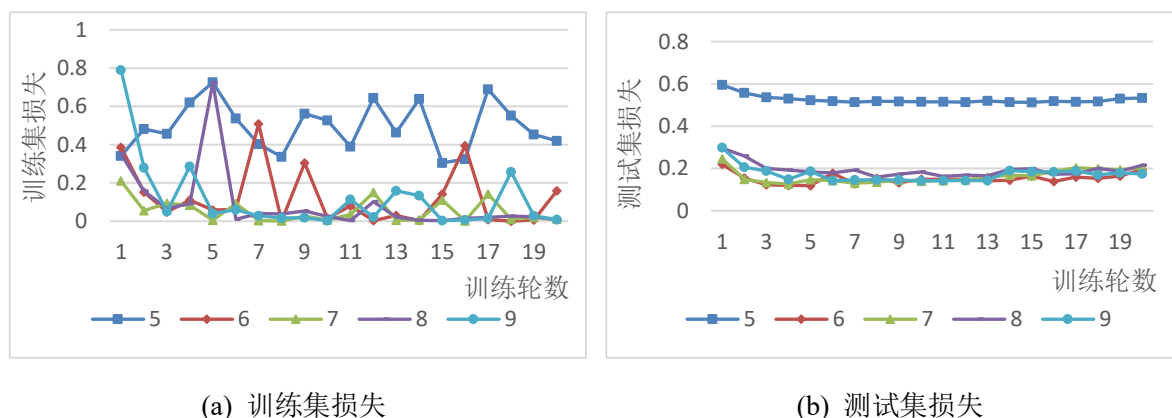


图 5-1 训练集损失和测试集损失 ($k=3$)

图 5-2 是卷积核尺寸 $k=3$ 时的精确率和召回率，横轴是训练轮数，共训练 20 轮，不同网络层数的数据由不同形状的折线表示。可以看到除了网络层数为 5 时双项指标都极不理想外，其他情况下的结果没有明显差异。精确率和测试集损失表现

出了大致相同的趋势：在 13 轮之后略有降低。这也验证了之前的猜想，训练轮数过大时模型确有过拟合的趋势。

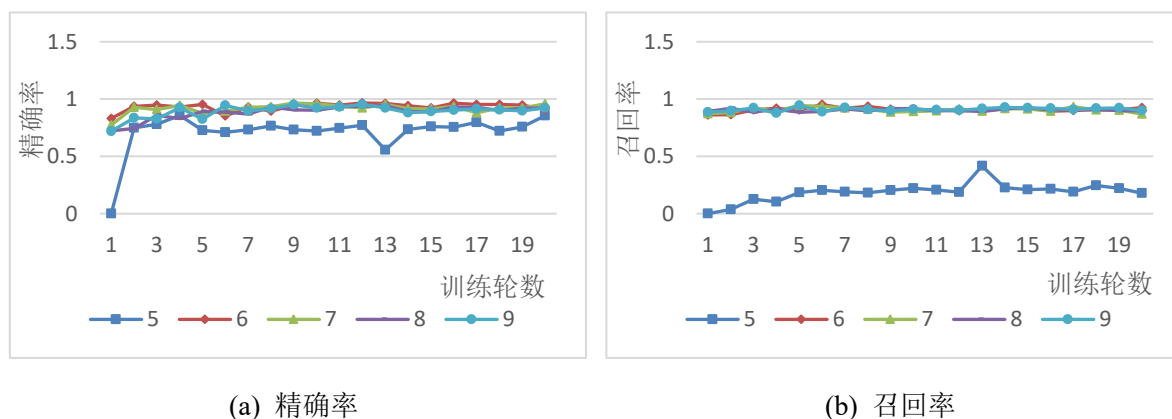


图 5-2 精确率和召回率 ($k=3$)

图 5-3 是卷积核尺寸 $k=3$ 时的误报率，横轴是训练轮数，共训练 20 轮，不同网络层数的数据由不同形状的折线表示。可以看到其趋势符合前面四项指标的结果：除网络层数为 5 时性能不理想外，其他情况下结果相差不大，或者说性能和网络层数没有明显的正相关性，而训练轮数超过 13 轮时误报率有上升趋势。

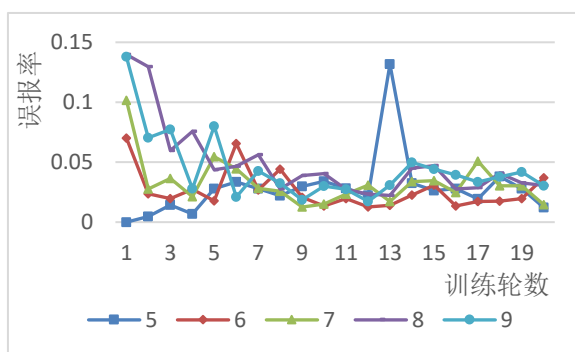


图 5-3 误报率 ($k=3$)

此时将关注点转回感受野和序列长度的关系上。在表 3-1 中可以查阅到当 $k=3$ 、 $l=5$ 时，感受野为 61，这远远小于序列长度 200，以上实验验证了当感受野远小于序列长度时，神经网络是无法做出合理预测的。然而当网络层数由 5 增加到 6 时，网络的预测性能迎来了质的飞跃，此时感受野为 127，尽管仍小于序列长度，但是网络已经有能力做出合理的预测了。更值得注意的是，当更深的网络出现过拟合时，

6层的网络反而在抵抗过拟合时稍有优势。当训练轮数超过13时，浅层网络(1=6)的测试集损失、精确率以及误报率领先于其他网络。这可能是由于浅层网络的表达能力有限，没有足够的“能力”过拟合。当卷积核尺寸 $k=5$ 时的情形与 $k=3$ 相似，该部分相关实验结果置于图 5-4 到图 5-6。区别只是在于凭借更大的卷积核，当网络深度为5时网络已经有足够的能力进行预测，此时感受野大小为125，而网络深度为4时仍然会欠拟合，此时感受野大小为63。

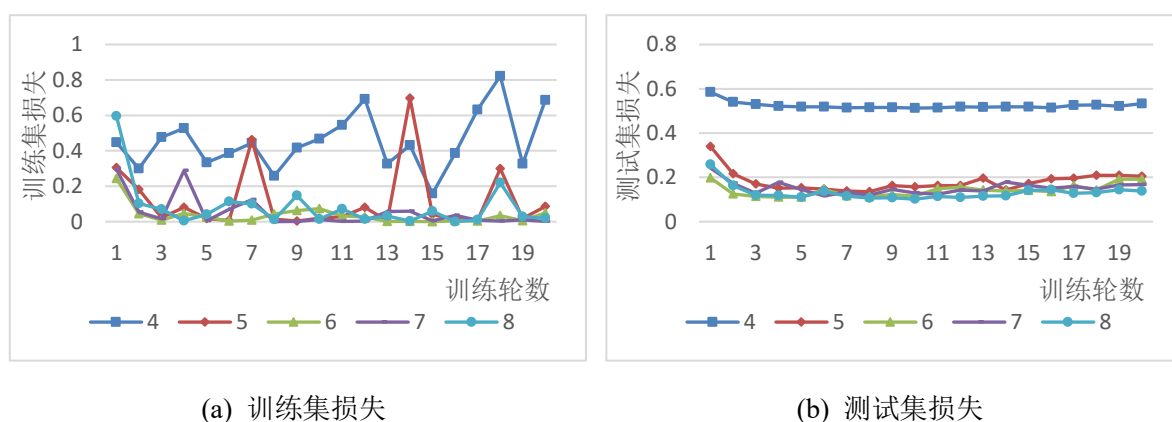


图 5-4 训练集损失和测试集损失 ($k=5$)

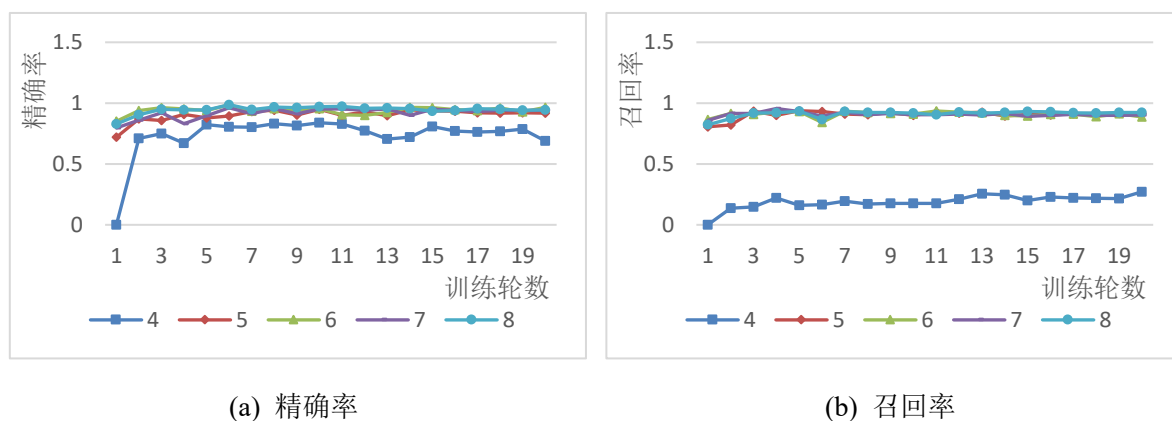


图 5-5 精确率和召回率 ($k=5$)

接下来研究卷积核尺寸 $k=7$ 时的情况。图 5-7 是 $k=7$ 时的训练集损失和测试集损失，横轴表示训练轮数，不同网络层数的数据由不同形状的折线表示。可以看到，除网络层数 $l=4$ 时欠拟合外，网络更深的情况下测试集损失并没有明显的降低。而且值得注意的是，由于大尺寸的卷积核，过拟合的临界点比 $k=3$ 和 $k=5$ 时来的更早，

可以看到在第 8 轮时，测试集损失略微出现反升，第 10 轮开始测试集损失明显增大。

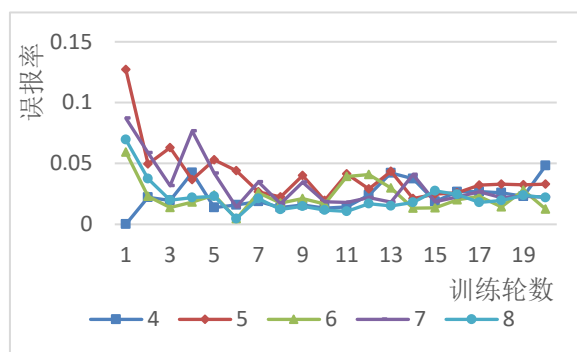
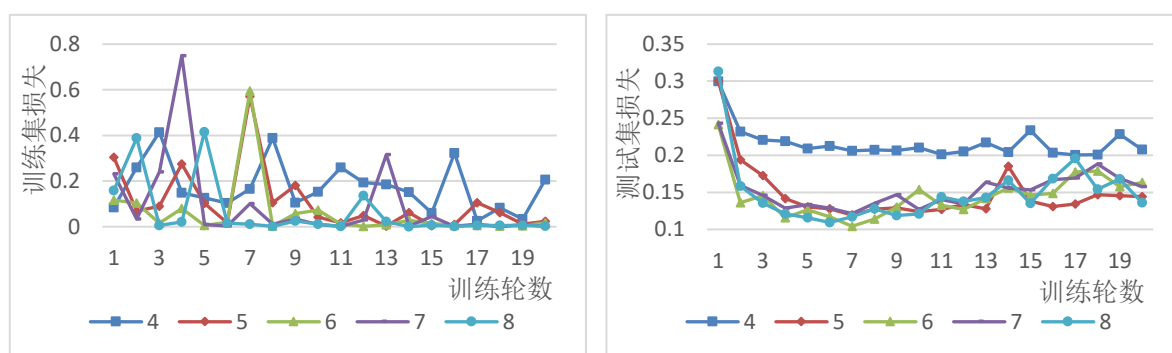


图 5-6 误报率 ($k=5$)

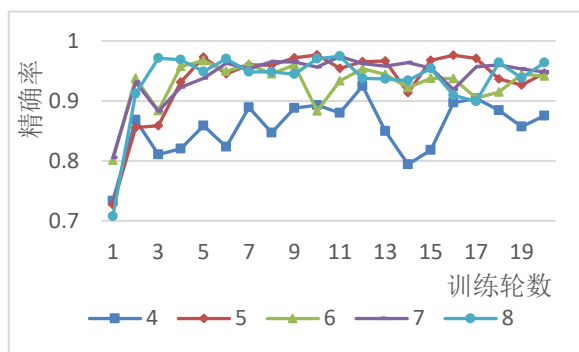


(a) 训练集损失

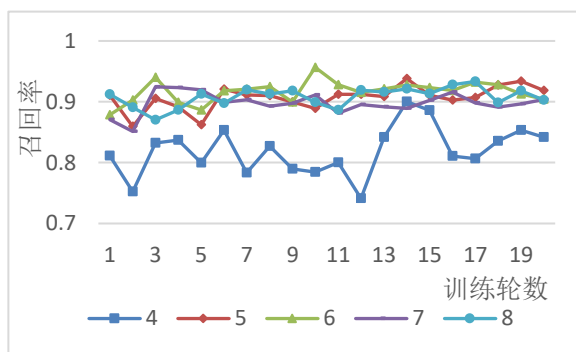
(b) 测试集损失

图 5-7 训练集损失和测试集损失 ($k=7$)

图 5-8 和图 5-9 分别是卷积核尺寸 $k=7$ 时的精确率、召回率和误报率曲线，横轴表示训练轮数，不同网络层数的数据由不同形状的折线表示。由于更大的卷积核，在相同网络深度时，网络的参数规模相对小卷积核时更大，所以过大参数规模网络的劣势会体现的更加明显。在图中体现为，在 11 轮左右，网络便呈现出明显的过拟合趋势：精确率和召回率降低、误报率提高。 $k=7$ 时三项实验指标的结果恰好验证了之前的结论：网络深度足够时，更深的网络没有明显的优势，而且更深的网络会导致过拟合的提前到来。 $k=9$ 时的结论也是类似的，相关实验结果置于图 5-10 到图 5-12。



(a) 精确率



(b) 召回率

图 5-8 精确率和召回率 (k=7)

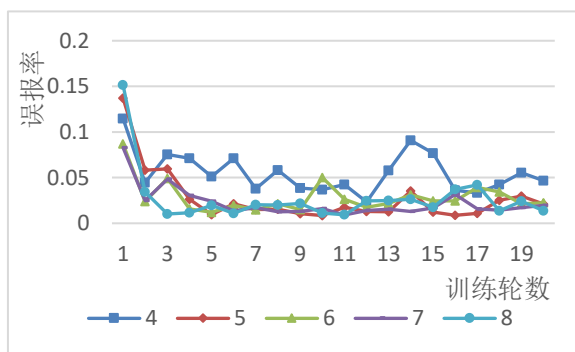
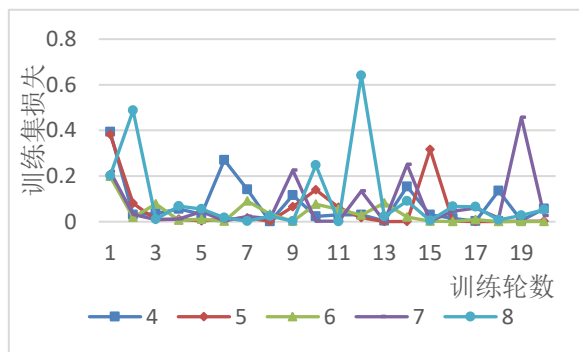
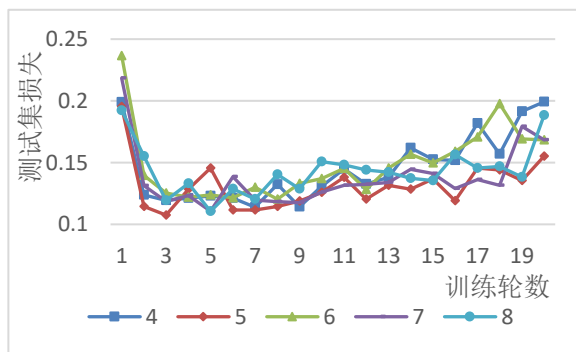


图 5-9 误报率 (k=7)



(a) 训练集损失



(b) 测试集损失

图 5-10 训练集损失和测试集损失 (k=9)

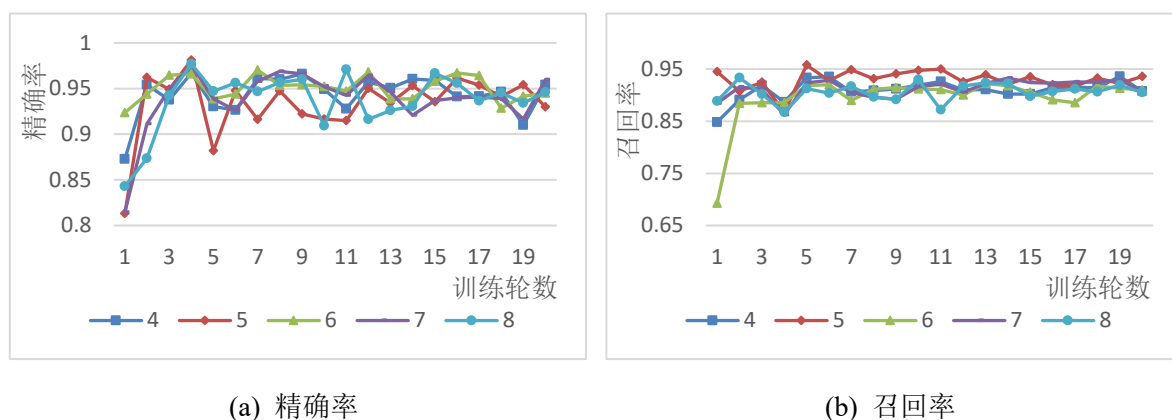


图 5-11 精确率和召回率 (k=9)

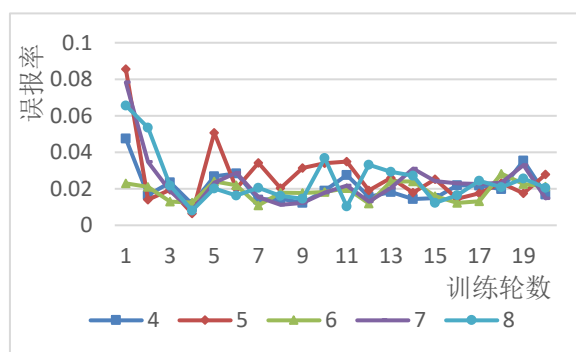


图 5-12 误报率 (k=9)

将各种超参数下的最佳结果列于表 5-3 到表 5-5 中，注意所取的“最佳”结果并不是该项指标在所有训练轮次中出现的最佳值，而是综合最佳结果所属轮次中该项的值，所以加粗的“最佳”结果在表中并不是最佳数值是正常的。定义综合最佳为精确率与召回率的和减去误报率，用公式表示为：

$$c = \text{precision} + \text{recall} - \text{fpr}$$

在表中将最佳结果的轮次标注于括号中并把综合最佳结果加粗显示。可以看到，在网络表达能力足够的前提下，更大规模的参数带来的数值提升微乎其微，相对于为了更大参数规模付出的计算资源来说，得到的微小的准确度的提升是无法与之匹配的。甚至可以得出更进一步的结论：在调整网络超参数时，使得感受野与序列长度相近即可，过大的网络规模不仅耗费计算资源，而且带来的微小提升也远低于研究人员的预期。

表 5-3 精确率 (mixed)

卷积核 网络层数	3	5	7	9
4	None	83.9 (10)	88.4 (18)	96.6 (9)
5	85.5 (20)	94.9 (10)	97.6 (16)	96.2 (16)
6	96.3 (13)	95.3 (4)	96.1 (7)	96.8 (12)
7	96.6 (9)	95.5 (8)	95.6 (10)	96.4 (12)
8	94.0 (12)	96.9 (8)	97.0 (10)	96.7 (15)

表 5-4 召回率 (mixed)

卷积核 网络层数	3	5	7	9
4	None	17.7 (10)	83.5 (18)	91.2 (9)
5	18.0 (20)	89.9 (10)	90.3 (16)	92.0 (16)
6	90.6 (13)	92.8 (4)	92.1 (7)	90.0 (12)
7	88.7 (9)	90.6 (8)	91.1 (10)	90.6 (12)
8	90.0 (12)	92.4 (8)	89.9 (10)	89.8 (15)

表 5-5 误报率 (mixed)

卷积核 网络层数	3	5	7	9
4	None	1.3 (10)	4.3 (18)	1.2 (9)
5	1.2 (20)	2.0 (10)	0.9 (16)	1.5 (16)
6	1.4 (13)	1.8 (4)	1.5 (7)	1.2 (12)
7	1.3 (9)	1.7 (8)	1.6 (10)	1.3 (12)
8	2.4 (12)	1.2 (8)	1.1 (10)	1.2 (15)

CWE-119 和 CWE-399 数据集的结果记录在表 5-6 到表 5-11 中。由于各项结果指标变化趋势与混合数据集类似，相关折线图不再列出。表中可以看到由于 CWE-399 数据集相对较小，所以同规模网络下，其结果的各项指标相对其他两种数据集更好。

表 5-6 精确率 (CWE-119)

卷积核 网络层数	3	5	7	9
4	None	83.0 (18)	95.7 (15)	96.0 (9)
5	76.0 (18)	93.8 (7)	96.3 (16)	94.3 (18)
6	94.9 (11)	95.0 (14)	94.6 (15)	95.6 (6)
7	94.8 (11)	92.0 (14)	94.5 (18)	96.7 (8)
8	95.0 (19)	96.0 (14)	97.1 (14)	95.6 (9)

表 5-7 召回率 (CWE-119)

卷积核 网络层数	3	5	7	9
4	None	14.3 (18)	63.0 (15)	87.4 (9)
5	14.8 (18)	89.8 (7)	87.2 (16)	88.0 (18)
6	84.8 (11)	88.1 (14)	88.1 (15)	87.8 (6)
7	84.9 (11)	85.3 (14)	86.9 (18)	85.8 (8)
8	86.3 (19)	83.3 (14)	87.3 (14)	87.8 (9)

最后，将 Li 等人 LSTM 模型的最佳结果与以上实验结果作对比，列于表 5-12 中。可以看到 CNN 模型在三种数据集的所有指标中全部优于 LSTM 模型。其中混合数据集的最佳结果由卷积核尺寸 $k=5$ ，网络深度 $l=8$ 的网络提供，CWE-119 数据集的最佳结果由卷积核尺寸 $k=7$ ，网络深度 $l=8$ 的网络提供，CWE-399 数据集的最佳结果由卷积核尺寸 $k=5$ ，网络深度 $l=6$ 的网络提供。

表 5-8 误报率 (CWE-119)

卷积核 网络层数	3	5	7	9
4	None	1.1 (18)	1.0 (15)	1.3 (9)
5	1.6 (18)	2.0 (7)	1.2 (16)	2.0 (18)
6	1.7 (11)	1.8 (14)	1.8 (15)	1.5 (6)
7	1.7 (11)	2.7 (14)	1.8 (18)	1.1 (8)
8	1.6 (19)	1.2 (14)	0.9 (14)	1.4 (9)

表 5-9 精确率 (CWE-399)

卷积核 网络层数	3	5	7	9
4	None	92.7 (20)	95.0 (20)	98.5 (16)
5	96.3 (13)	98.1 (18)	98.6 (10)	98.8 (19)
6	98.5 (19)	98.4 (19)	98.8 (9)	98.8 (19)
7	97.3 (14)	98.1 (18)	99.1 (9)	98.8 (9)
8	97.7 (20)	97.4 (12)	97.9 (9)	98.5 (15)

表 5-10 召回率 (CWE-399)

卷积核 网络层数	3	5	7	9
4	None	19.2 (20)	95.6 (20)	97.9 (16)
5	90.6 (13)	97.0 (18)	97.1 (10)	98.1 (19)
6	96.5 (19)	98.1 (19)	97.7 (9)	97.0 (19)
7	96.6 (14)	96.9 (18)	96.8 (9)	96.9 (9)
8	96.7 (20)	96.2 (12)	98.3 (9)	97.4 (15)

表 5-11 误报率 (CWE-399)

卷积核 网络层数	3	5	7	9
4	None	0.7 (20)	2.5 (20)	0.7 (16)
5	1.4 (13)	0.9 (18)	0.7 (10)	0.6 (19)
6	0.7 (19)	0.8 (19)	0.6 (9)	0.5 (19)
7	1.3 (14)	0.9 (18)	0.4 (9)	0.6 (9)
8	1.2 (20)	1.3 (12)	1.0 (9)	0.7 (15)

表 5-12 LSTM 模型与 CNN 模型结果对比

指标 模型	精确率	召回率	误报率
LSTM(mixed)	86.9	83.9	5.1
CNN(mixed)	96.9	92.4	1.2
LSTM(CWE-119)	91.7	82.0	2.9
CNN(CWE-119)	97.1	87.3	0.9
LSTM(CWE-399)	94.6	95.3	2.8
CNN(CWE-399)	98.8	98.1	0.6

需要说明的是，这些最佳结果是在将综合指标对比到小数点后两位才得出的，实际情况是在网络的感受野足够的前提下，更大参数规模的网络消耗了更多的计算资源，却无法带来与消耗相匹配的准确度提升。如果将模型应用于实际生产中，只要感受野与数据序列长度相当或使感受野比数据长度稍大即可，此时可兼顾准确率与效率。

代码漏洞检测模型基于时序卷积网络，并结合多种现代技术提高了网络的性能和稳定性，与循环网络相比，该模型在理论上有更多的优势。首先是计算的可并行化。在循环网络中，后面时间步的预测必须等待其前任完成，而卷积运算由于在每层使用相同的卷积核，可以并行完成卷积。因此，在训练和预测中，卷积网络可以

将整个输入序列同时处理，而不是像循环网络中按照时间顺序依次处理。然后是更灵活的感受野尺寸。时序卷积网络有多种方式改变其感受野尺寸。比如堆叠更多的扩张卷积层，或者使用更大的扩张系数，或者加大卷积核的尺寸。因此它可以更好的控制模型的内存消耗，并且灵活适应不同领域的任务。接着是更稳定的梯度。与循环结构不同，卷积网络具有与序列的时间方向不同的反向传播路径，所以相比循环网络而言不容易出现梯度爆炸和梯度消失问题，而且通过使用 ReLU 激活函数和残差单元，使训练的梯度更加稳定。最后是训练时更低的内存消耗。在处理长输入序列时，LSTM 和 GRU 需要消耗大量的内存来存储其多个单元的部分结果。但是在卷积网络中，一个层中共享所有卷积核，反向传播路径长度仅取决于网络深度。因此在实践中，卷积网络的内存消耗远小于循环网络。

5.4 本章小结

本章首先介绍了实验环境和采用的实验数据，然后对代码漏洞检测网络的规模参数进行了实验与研究。通过对比各种参数的变化与指标的训练趋势，得出结论：在网络的感受野足够的前提下，更大参数规模的网络消耗了更多的计算资源，却无法带来与消耗相匹配的准确度提升。最后将 CNN 网络的最佳结果与 LSTM 网络作对比，证明 CNN 网络在三种数据集的各项指标中均优于 LSTM 网络。

6 总结与展望

6.1 工作总结

互联网行业的飞速发展导致每年的软件代码量激增，尽管企业和开发人员在软件安全编程和漏洞检测上投入了大量的时间和精力，但面对日益增长的巨大的代码基数，代码漏洞检测仍然是行业内亟需解决的重要问题。当前的静态检测和动态检测方案都严重依赖于专家经验和人工操作，这不仅无法满足社会对于软件安全的要求，也与计算机软件自动化的理念背道而驰。本文建立了一个基于卷积深度神经网络的代码漏洞检测方案，包含了从代码预处理到模型构建的一系列方法，可以实现基于大数据的代码漏洞自动化检测。

首先提出了一种将程序代码转换为单词向量序列的方案，在兼顾程序语义及易用性的前提下将程序代码段转换为神经网络的输入数据。其中包括对原始代码的切割操作，以保留程序的语义信息并精简代码结构，接着使用词法分析器将代码编码为向量，并生成向量序列。还介绍了数据加载的相关工作，其中数据加载器作为数据与网络的中间层起到了重要的承接作用。

然后建立了基于卷积神经网络的代码漏洞检测模型。为了使神经网络适合于处理长序列的代码检测任务，在模型中结合了诸多当下流行且有效的技术，如因果卷积、扩张卷积以及残差单元等。因果卷积使得网络得以按照时序处理输入，防止了信息从过去向未来的泄漏；扩张卷积使网络在获得足够的感受野的前提下，能够尽量降低网络层数，从而使网络的参数规模得到有效的控制；而残差单元则是稳定梯度和加快训练速度的利器。

最后经过分析研究实验结果，给出了兼顾效率与准确度的网络超参数调整原则，使模型可以适配于更多的数据集。在实验中主要研究了超参数对于检测结果的影响，并从准确率、召回率、误报率、漏报率等多个指标评估了各种情况下的实验效果。结果表明，代码漏洞检测网络在当前代码数据集上取得了最好的结果，优于 LSTM

网络。这对于卷积网络在时间序列任务上的应用是一个极佳的案例，也引发了何种网络更适合于序列任务的思考。

6.2 工作展望

在代码预处理工作中，通过将代码处理成令牌，进而映射到低维空间的思路，对于单一语言来说是简单而有效的，然而这种方法导致网络无法直接适用于其他的语言。如果希望把模型用于分析其他语言，就需要全新的代码预处理程序，这限制了模型的可扩展性。而且在高级程序语言中，一条语句可以表示一个运算、多个操作、甚至是类的定义，所以不同语言的单一语句表达的语义千差万别。从另一个极端考虑，如果处理对象是机器代码，尽管脱离了源程序语言类型的限制，却陷入了硬件平台的限制，因为机器代码中包含了硬件专用的指令集，并与硬件的特性相关，比如寄存器和存储器架构。所以如果能生成一种中间表示，既与编程语言无关，又与硬件平台无关，还有着固定的语法规则，可以保留程序的语义，就可以作为一种理想的输入数据。

由于当前的数据集仅有两种漏洞类型，所以当前实验只做了二分类(正常/异常)的预测。当使用同一个模型处理更多类型的数据集时，随着漏洞类型的丰富，继续采用二分类的预测可能不是一种好的选择，而多分类可以提供更好的结果。例如一种简明的思路就是，每种漏洞类型作为一种异常类。这使得模型不仅可以判断是否有漏洞，而且进一步给出漏洞的类型，这对于漏洞的修复无疑是更好的补充信息。

鉴于卷积网络模型在代码漏洞检测上的成果，甚至可以进一步考虑将其应用于代码修复。代码修复是一项序列生成工作，可以使用生成对抗网络（Generative Adversarial Networks，GAN），而且神经网络机器翻译模型（Neural Machine Translation，NMT）已经是非常成熟的代码修复方法，可以作为 GAN 的生成器部分，而判别器就由卷积网络模型来承担。通过生成器和判别器的对抗训练，可以进一步提高生成器的性能和判别器的准确度，从而达到代码修复的目的。

致谢

短暂的三年研究生生活一晃而过，如今到了说再见的时刻，不禁感慨万千。

首先要感谢我的导师付才，付才老师是一个对待工作极其严谨而认真的人，对学生也有着高要求，我们的点点进步都离不开付才老师的悉心指导和耐心督促。还要感谢韩兰胜老师和刘铭老师，两位老师生活中对我们关怀备至，学习上为我们答疑解惑，感谢老师们的辛勤付出。

感谢实验室的同学们，不管是在科研项目中大放异彩的郑玲玲同学，还是畅游于学术海洋中的彭晨晨同学，抑或是一起喝酒吃肉的杨成龙同学和夏攀同学，大家都有值得我学习的优秀品质。还要特别感谢实验室的老师姐——杨佳博士，作为实验室资历最深的同学，杨佳博士在生活和学习中给予我们很多的帮助，还不断为我们制造快乐，让我们的科研生活充满了欢声笑语。

感谢智慧华中大正版软件服务平台，让我可以免费使用各种优秀的正版软件，包括但不限于 Windows 操作系统、Office 办公软件、MATLAB，如果没有它，我将耗费大量的资金在软件上，或是成为盗版软件的受害者。

感谢我的笔记本电脑，在我实验的过程中没有出岔子，感谢微软优秀的操作系统和办公软件，在我写论文的过程中没有崩溃。它们让我不必费心思在软硬件问题上，从而可以专心学术。

这次毕业，我们将告别 22 年的校园生活，走到广袤的社会舞台中，愿每个人今后都能在自己的行业和岗位上发光发热，为祖国的建设添砖加瓦。

参考文献

- [1] Chung J, Gulcehre C, Cho K et al. Empirical Evaluation of Gated Recurrent Neural Networks On Sequence Modeling. arXiv preprint arXiv:1412.3555, 2014.
- [2] Pascanu R, Gulcehre C, Cho K et al. How to Construct Deep Recurrent Neural Networks. arXiv preprint arXiv:1312.6026, 2013.
- [3] Jozefowicz R, Zaremba W, Sutskever I. An Empirical Exploration of Recurrent Network Architectures. In, ed. International Conference on Machine Learning. 2015. 2342-2350
- [4] Greff K, Srivastava R K, Koutník J et al. LSTM: A Search Space Odyssey. ArXiv. Org. In, edMarch, 2015.
- [5] Zhang S, Wu Y, Che T et al. Architectural Complexity Measures of Recurrent Neural Networks. In, ed. Advances in neural information processing systems. 2016. 1822-1830
- [6] Melis G, Dyer C, Blunsom P. On the State of the Art of Evaluation in Neural Language Models. arXiv preprint arXiv:1707.05589, 2017.
- [7] Lecun Y, Boser B, Denker J S et al. Backpropagation Applied to Handwritten Zip Code Recognition. Neural Comput, 1989, 4: 541-551
- [8] Hinton G E. Connectionist Learning Procedures. In, ed. Machine learning. Elsevier, 1990. 555-610
- [9] Waibel A, Hanazawa T, Hinton G et al. Phoneme Recognition Using Time-Delay Neural Networks. Backpropagation: Theory, Architectures and Applications, 1995, 35-61
- [10] Bottou L, Fogelman Soulié F, Blanchet P et al. Speaker-Independent Isolated Digit Recognition: Multilayer Perceptrons Vs. Dynamic Time Warping. Neural Networks, 1990, 4: 453-465
- [11] Kalchbrenner N, Grefenstette E, Blunsom P. A Convolutional Neural Network for

- Modelling Sentences. CoRR, 2014.
- [12] Kim Y. Convolutional Neural Networks for Sentence Classification. CoRR, 2014.
- [13] Conneau A, Schwenk H, Barrault L I C et al. Very Deep Convolutional Networks for Natural Language Processing. CoRR, 2016.
- [14] Johnson R, Zhang T. Effective Use of Word Order for Text Categorization with Convolutional Neural Networks. CoRR, 2014.
- [15] Kalchbrenner N, Espeholt L, Simonyan K et al. Neural Machine Translation in Linear Time. CoRR, 2016.
- [16] Gehring J, Auli M, Grangier D et al. A Convolutional Encoder Model for Neural Machine Translation. CoRR, 2016.
- [17] van den Oord A A R, Dieleman S, Zen H et al. WaveNet: A Generative Model for Raw Audio. CoRR, 2016.
- [18] Dam H K, Tran T, Pham T. A Deep Language Model for Software Code. CoRR, 2016.
- [19] Gu X, Zhang H, Zhang D et al. Deep API Learning. In: AcmACM ed Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering. FSE 2016. New York, NY, USA: 2016. 631-642
- [20] Levy D, Wolf L. Learning to Align the Source Code to the Compiled Object Code. In: Org JJMLR.org ed Proceedings of the 34th International Conference on Machine Learning - Volume 70. ICML'17. 2017. 2043-2051
- [21] Cummins C, Petoumenos P, Wang Z et al. End-To-End Deep Learning of Optimization Heuristics. In, ed. 2017 26th International Conference on Parallel Architectures and Compilation Techniques (PACT). IEEE, 2017. 219-232
- [22] Allamanis M, Barr E T, Bird C et al. Suggesting Accurate Method and Class Names. In: AcmACM ed Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering. ESEC/FSE 2015. New York, NY, USA: 2015. 38-49
- [23] Alon U, Zilberstein M, Levy O et al. Code2vec: Learning Distributed Representations

- of Code. Proceedings of the ACM on Programming Languages, 2019, POPL: 40
- [24] White M, Tufano M, Vendome C et al. Deep Learning Code Fragments for Code Clone Detection. In: AcmACM ed Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering. ASE 2016. New York, NY, USA: 2016. 87-98
- [25] Raychev V, Vechev M, Yahav E. Code Completion with Statistical Language Models. In: AcmACM ed Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation. PLDI '14. New York, NY, USA: 2014. 419-428
- [26] Yang Y, Jiang Y, Gu M et al. A Language Model for Statements of Software Code. 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE). 2017. 682-687
- [27] Allamanis M, Peng H, Sutton C. A Convolutional Attention Network for Extreme Summarization of Source Code. In, ed. International Conference on Machine Learning. 2016. 2091-2100
- [28] Mou L, Li G, Zhang L et al. Convolutional Neural Networks Over Tree Structures for Programming Language Processing. In, ed. Thirtieth AAAI Conference on Artificial Intelligence. 2016.
- [29] Hovsepyan A, Scandariato R, Joosen W et al. Software Vulnerability Prediction Using Text Analysis Techniques. In: AcmACM ed Proceedings of the 4th International Workshop on Security Measurements and Metrics. MetriSec '12. New York, NY, USA: 2012. 7-10
- [30] Pang Y, Xue X, Namin A S. Predicting Vulnerable Software Components through N-Gram Analysis and Statistical Feature Selection. 2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA). 2015. 543-548
- [31] Abou-Assaleh T, Cercone N, Keselj V et al. N-Gram-Based Detection of New Malicious Code. Proceedings of the 28th Annual International Computer Software

- and Applications Conference, 2004. COMPSAC 2004. 2004. 41-42
- [32] Cesare S, Xiang Y. Malware Variant Detection Using Similarity Search over Sets of Control Flow Graphs. 2011IEEE 10th International Conference on Trust, Security and Privacy in Computing and Communications. 2011. 181-189
- [33] Kolter J Z, Maloof M A. Learning to Detect Malicious Executables in the Wild. In: AcmACM ed Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. KDD '04. New York, NY, USA: 2004. 470-478
- [34] Shabtai A, Moskovitch R, Elovici Y et al. Detection of Malicious Code by Applying Machine Learning Classifiers On Static Features: A State-Of-The-Art Survey. Information Security Technical Report, 2009, 1: 16-29
- [35] Hsiao C, Cafarella M, Narayanasamy S. Using Web Corpus Statistics for Program Analysis. In: AcmACM ed Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications. OOPSLA '14. New York, NY, USA: 2014. 49-65
- [36] Raychev V, Vechev M, Krause A. Predicting Program Properties From "Big Code". In: AcmACM ed Proceedings of the 42Nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages. POPL '15. New York, NY, USA: 2015. 111-124
- [37] White M, Vendome C, Linares-V A Squez M et al. Toward Deep Learning Software Repositories. In: Press IEEE Press ed Proceedings of the 12th Working Conference on Mining Software Repositories. MSR '15. Piscataway, NJ, USA: 2015. 334-345
- [38] Chua Z L, Shen S, Saxena P et al. Neural Nets Can Learn Function Type Signatures From Binaries. In, edVancouver, BC: USENIX Association, 2017. 99-116
- [39] Yamaguchi F, Lottmann M, Rieck K. Generalized Vulnerability Extrapolation Using Abstract Syntax Trees. In: AcmACM ed Proceedings of the 28th Annual Computer Security Applications Conference. ACSAC '12. New York, NY, USA: 2012. 359-368

- [40] Le Goues C, Nguyen T, Forrest S et al. GenProg: A Generic Method for Automatic Software Repair. *IEEE Trans. Softw. Eng.* January 2012, 2012, 1: 54-72
- [41] Long F, Rinard M. Automatic Patch Generation by Learning Correct Code. *SIGPLAN Not.* January 2016, 2016, 1: 298-312
- [42] Gupta R, Pal S, Kanade A et al. Deepfix: Fixing Common C Language Errors by Deep Learning. *Thirty-First AAAI Conference on Artificial Intelligence.* 2017.
- [43] Harer J, Ozdemir O, Lazovich T et al. Learning to Repair Software Vulnerabilities with Generative Adversarial Networks. *CoRR*, 2018.
- [44] Li Z, Zou D, Xu S et al. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. *CoRR*, 2018.
- [45] Latoza T D, Venolia G, Deline R. Maintaining Mental Models: A Study of Developer Work Habits. In, ed. *Proceedings of the 28th international conference on Software engineering.* ACM, 2006. 492-501
- [46] Yadron D. After Heartbleed Bug, a Race to Plug Internet Hole. *Wall Street Journal*, 2014.
- [47] Arnold C. After Equifax Hack, Calls for Big Changes in Credit Reporting Industry. In, ed2017.
- [48] Xu Z A K T. A Memory Model for Static Analysis of C Programs. In: Heidelberg S BSpringer Berlin Heidelberg ed *Leveraging Applications of Formal Methods, Verification, and Validation.* Berlin, Heidelberg: 2010. 535-548
- [49] King J C. Symbolic Execution and Program Testing. *Commun. ACM* July 1976, 1976, 7: 385-394
- [50] Allamanis M, Barr E T, Devanbu P et al. A Survey of Machine Learning for Big Code and Naturalness. *ACM Computing Surveys (CSUR)*, 2018, 4: 81
- [51] Allamanis M, Brockschmidt M. Smartpaste: Learning to Adapt Source Code. *arXiv preprint arXiv:1705.07867*, 2017.
- [52] Park E, Cavazos J, Alvarez M A. Using Graph-Based Program Characterization for

- Predictive Modeling. In, ed. Proceedings of the Tenth International Symposium on Code Generation and Optimization. ACM, 2012. 196-206
- [53] Mikolov T, Sutskever I, Chen K et al. Distributed Representations of Words and Phrases and their Compositionality. In: Burges C J C, Bottou L, Welling M et al, ed. Advances in Neural Information Processing Systems 26. Curran Associates, Inc., 2013. 3111-3119
- [54] Elman J L. Finding Structure in Time. Cognitive Sci, 1990, 2: 179-211
- [55] Werbos P J. Backpropagation through Time: What It Does and How to Do It. P Ieee, 1990, 10: 1550-1560
- [56] Bengio Y, Simard P, Frasconi P. Learning Long-Term Dependencies with Gradient Descent is Difficult. IEEE Transactions on Neural Networks, 1994, 2: 157-166
- [57] Pascanu R, Mikolov T, Bengio Y. Understanding the Exploding Gradient Problem. CoRR, 2012.
- [58] Hochreiter S, Schmidhuber J U R. Long Short-Term Memory. Neural Comput. November 15, 1997, 1997, 8: 1735-1780
- [59] Cho K, van Merriënboer B, Bahdanau D et al. On the Properties of Neural Machine Translation: Encoder-Decoder Approaches. CoRR, 2014.
- [60] Goodfellow I, Bengio Y, Courville A. Deep Learning. MIT press, 2016.
- [61] He K, Zhang X, Ren S et al. Deep Residual Learning for Image Recognition. In, ed. Proceedings of the IEEE conference on computer vision and pattern recognition. 2016. 770-778
- [62] He K, Sun J. Convolutional Neural Networks at Constrained Time Cost. The IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2015.